Stabilisation des Modèles Basés sur l'Énergie grâce à la Contrainte de Lipschitz

Angel LAGRANGE

Rapport de Projet de Fin d'Études (PFE)

Établissement d'accueil : ANITI – Équipe DEEL, IRT Saint Exupéry, Toulouse
Encadrants : Victor BOUTIN (CNRS)
Franck MAMALET (IRT Saint Exupéry)
Tutrice école : Sylvie CHAMBON (ENSEEIHT)



25 août 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble des personnes qui ont contribué à la réalisation de ce stage et à la réussite de ce projet.

Je remercie chaleureusement M. Victor BOUTIN et M. Franck MAMALET, mes encadrants de stage au sein de l'équipe DEEL, pour leur accompagnement, leurs conseils, leur bienveillance et leur disponibilité tout au long de ce stage.

Je suis particulièrement reconnaissant à Mme. Sylvie CHAMBON, M. Ronan GUIVARCH et à l'ENSEEIHT de m'avoir donné l'opportunité de réaliser ce stage au sein d'ANITI. Cette expérience de recherche, extrêmement formatrice et enrichissante, a renforcé mon envie de poursuivre dans cette voie en entamant une thèse.

Enfin, je souhaite remercier l'ensemble des membres de l'équipe DEEL, d'ANITI et de l'IRT Saint Exupéry pour leur accueil et les échanges enrichissants qui ont rythmé ces six mois de collaboration.

Table des matières

1 Contexte et Enjeux **4**
- 1.1 Du cerveau aux modèles bio-inspirés . . . 4
- 1.2 Un appétit computationnel croissant . . . 6

2 Introduction et Travaux connexes **7**
- 2.1 Modèles Basés sur l'Énergie (EBM) . . . 10
  - 2.1.1 Entraîner un EBM : Maximum de vraisemblance et divergence contrastive 10
- 2.2 Échantillonnage MCMC par dynamique de Langevin . . . 12
  - 2.2.1 Dynamique de Langevin non-ajustée (ULA, Unadjusted Langevin Algorithm) 12
  - 2.2.2 MCMC à courte durée . . . 13
  - 2.2.3 Dynamique de Langevin ajustée (MALA, Metropolis-Adjusted Langevin Algorithm) . . . 13
  - 2.2.4 Buffer Replay . . . 14
- 2.3 Limitations . . . 14
  - 2.3.1 Instabilité numérique . . . 14
  - 2.3.2 Problèmes de convergence . . . 15
  - 2.3.3 Trajet Bruit → Distribution : Le défi de la haute dimension . . . 16
  - 2.3.4 Paysage énergétique . . . 16
- 2.4 La contrainte de Lipschitz . . . 17
- 2.5 Réseaux de neurones et contrainte de Lipschitz . . . 18
  - 2.5.1 Principe général . . . 18
  - 2.5.2 Méthodes pour imposer la contrainte 1-Lipschitz . . . 18
  - 2.5.3 Extension à un réseau K-Lipschitz . . . 19

3 Etude des EBMs avec contraintes K-Lipschitz **19**
- 3.1 Hypothèses . . . 19
  - 3.1.1 Propriétés des EBMs K-Lipschitz . . . 19
  - 3.1.2 Lien avec la distance de Wasserstein . . . 20
  - 3.1.3 Le transport optimal et entraînement des EBM Lipschitz . . . 21

4 Pistes suivies **23**
- 4.1 Baseline non contrainte . . . 23
- 4.2 Apprentissage Lipschitz . . . 23
  - 4.2.1 La bibliothèque `deel-torchlip` . . . 23
- 4.3 Heuristique optimale pour déterminer K . . . 24
  - 4.3.1 Évaluation du nombre de pas de Langevin et critère d'arrêt d'entraînement 26
  - 4.3.2 Critère d'arrêt d'entraînement . . . 27
- 4.4 Architecture à double réseau avec pré-initialisation Lipschitz . . . 28
  - 4.4.1 Phase de pré-initialisation : modèle Lipschitz. . . . 29
  - 4.4.2 Phase principale : modèle non contraint. . . . 29
- 4.5 Semi-Lipschitz . . . 31
  - 4.5.1 Architecture . . . 31
  - 4.5.2 Découpage architectural . . . 32
  - 4.5.3 Choix de la profondeur contrainte L_1 . . . 32

5 Protocole expérimental **33**
5.1 Jeu de données . . . 33
5.2 Architecture . . . 33
5.2.1 Initialisation et réglages des hyper-paramètres . . . 33
5.3 Métriques . . . 34

6 Résultats et analyses **35**
6.1 Base line non contrainte . . . 35
6.2 Réseau Lipschitz . . . 36
6.3 1-Lipschitz . . . 36
6.3.1 K-Lipschitz . . . 37
6.3.2 $K_{Optimal}$-Lipschitz . . . 38
6.4 Pré-initialisation Lipschitz . . . 40
6.5 Semi-Lipschitz . . . 40

7 Conclusion **42**

A Long-run et Buffer **47**

B Régularisation **48**

C DualHead : Architecture propre pour la pré-initialisation Lipschitz et HKR **49**
C.1 Description de l'architecture dual-head. . . . 49
C.2 La fonction de coût HKR. . . . 49
C.3 Inférence conjointe par pondération sigmoïde . . . 49
C.4 Résultats . . . 51
C.4.1 Nécessité d'un rééquilibrage des gradients . . . 51
C.4.2 Nécessité d'une régularisation de la tête EBM . . . 51
C.4.3 Comportement du buffer et compromis vitesse/qualité . . . 52

D Économie computationnelle **52**

E Hyper-paramètres **53**

F Bilan **54**

G Preuves **55**

1 Contexte et Enjeux

Ce rapport décrit mon stage de 3ème année de mon école d'ingénieur. Ce dernier s'inscrit dans le cadre d'ANITI, au sein de l'équipe DEEL (Dependable and Explainable Learning) de l'IRT Saint-Exupéry, en collaboration avec le CerCo (CNRS), sous la supervision de Victor Boutin (CerCo) et Franck Mamalet (IRT Saint-Exupéry).

1.1 Du cerveau aux modèles bio-inspirés

Le CerCo est spécialisé sur l'étude de différentes modalités sensorielles et de leur intégration, des fonctions cognitives telles que la mémoire, la reconnaissance de l'objet mais aussi la conscience et les états mentaux. C'est dans ce cadre que s'inscrit la question de recherche de Victor Boutin :

Comment le cerveau parvient-il à généraliser aussi bien à partir d'aussi peu de données ?

Il suffit par exemple d'observer un petit nombre de chiens ou de chats pour être ensuite capable de reconnaître de nombreuses variations de cet animal, races différentes, postures différentes, points de vue différents, sans jamais les avoir vues auparavant (Figure 1).

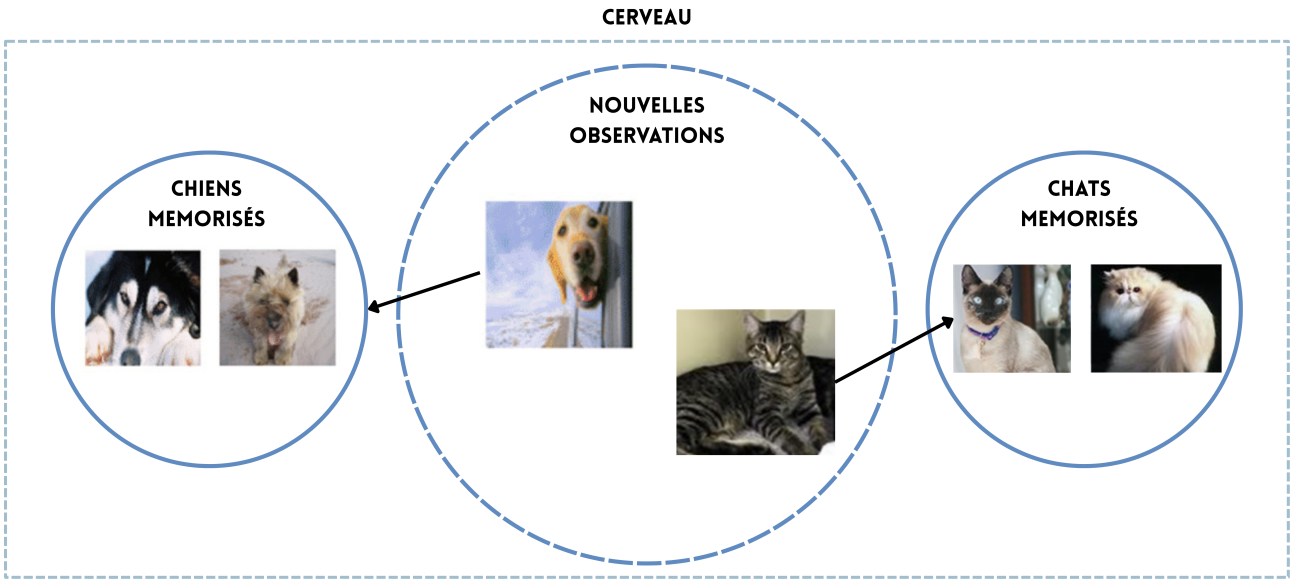


FIGURE 1 – Le cerveau généralise à partir de peu d'observations : après avoir mémorisé peu d'exemples de chiens et de chats, il est capable de reconnaître de nouvelles instances très différentes de ces catégories.

Face à cette observation, l'hypothèse de travail de Victor Boutin est que le cerveau implémente un **modèle génératif interne** du monde. Intuitivement, un modèle génératif peut être vu comme un simulateur qui capture les règles ayant produit les données observées, plutôt qu'une simple mémorisation d'exemples passés.

Connaître ces règles permet d'imaginer de nouveaux cas plausibles et de prédire ce que l'on n'a encore jamais observé, par exemple reconnaître un objet sous un nouveau point de vue après ne l'avoir vu qu'une seule fois (Figure 2).

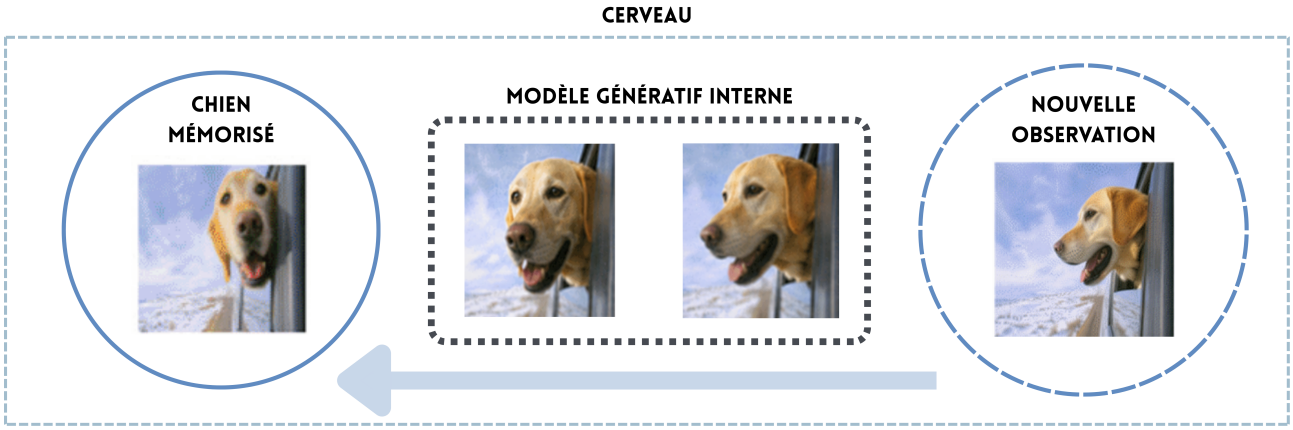


FIGURE 2 – Illustration d'un hypothétique modèle génératif interne : à partir d'une observation, le modèle génère des variations plausibles (ici, une nouvelle orientation de la tête) permettant de reconnaître ainsi un chien déjà mémorisé.

Pour tester cette hypothèse, Victor Boutin adopte une démarche **en trois étapes** :

1. **Développer** des théories mathématiques décrivant la façon dont le cerveau traite l'information
2. **Implémenter** ces théories sous forme de modèles d'apprentissage profond, en particulier des modèles génératifs
3. **Évaluer** ces modèles en les confrontant à des données comportementales et neuronales, afin de comparer leurs propriétés à celles du cerveau

C'est dans cette étape d'implémentation que s'inscrivent les *modèles basés énergie* (détaillés dans les sections suivantes), auxquels Victor Boutin consacre une partie de sa recherche. Cependant, ces modèles souffrent en pratique d'une instabilité d'entraînement qui limite leur usage.

C'est pourquoi la collaboration avec l'équipe DEEL est apparue. En effet, leur expertise porte sur les méthodes garantissant la robustesse certifiée des réseaux de neurones. Une partie des travaux de Franck Mamalet porte notamment sur des méthodes permettant de contrôler mathématiquement la sensibilité d'un réseau à de petites perturbations de son entrée, une propriété qui, comme nous le détaillerons, permet de garantir sa robustesse par construction.

Ainsi, les modèles issus de cette collaboration pourront bénéficier de deux atouts complémentaires : la fiabilité, apportée par les méthodes de robustesse de DEEL, et l'efficience, potentiellement héritée de leur similitude avec le fonctionnement du cerveau.

Cette seconde propriété ouvre une perspective plus lointaine : si de tels modèles, en intégrant des caractéristiques propres au cerveau, devenaient à la fois plus fiables et plus faciles à exploiter, ils pourraient, à terme, contribuer aux réflexions sur la réduction du coût computationnel de l'apprentissage profond.

1.2 Un appétit computationnel croissant

En effet, l'essor de l'IA moderne s'est accompagné d'un coût de plus en plus lourd, pas seulement financier, mais aussi environnemental. L'entraînement des grands modèles génératifs nécessite désormais des grandes quantités d'énergie et de ressources informatiques. Pour illustrer l'ampleur du problème : l'entraînement de GPT-3 seul a consommé environ 1287 mégawattheures d'électricité, produisant environ 552 tonnes de CO_2 [37], soit l'équivalent, d'un trajet d'une voiture classique de 3.2 millions de kilomètres [38]. Par exemple, pour entraîner nous-mêmes notre propre modèle GPT-3 avec un service GPU classique, il nous faudrait attendre 355 ans et payer 4 millions d'euros [27]. Pourtant GPT-3 est un vieux modèle et est plutôt petit pour sa catégorie ; GPT-3 possède 175 milliards de paramètres, GPT-4 est estimé à 666 milliards de paramètres et GPT-5.5 à 9 700 milliards [26]. Avec des modèles de cette échelle, il est facile d'imaginer une consommation de plusieurs millions de kilowattheures lors d'un seul cycle d'entraînement [51]. Du côté des infrastructures, la consommation électrique mondiale des centres de données est estimée à 415 TWh en 2024, et devrait presque tripler pour atteindre 945 TWh d'ici 2030, l'IA étant identifiée comme le principal moteur de cette explosion [51].

Dans ce contexte, la *fiabilité* des cycles d'entraînement devient une préoccupation critique. Chaque entraînement divergent, qui s'effondre ou qui nécessite un redémarrage à zéro, représente non seulement un gaspillage computationnel, mais aussi un coût environnemental direct. Une procédure d'entraînement fragile, difficile à régler et sujette aux instabilités oblige les praticiens à multiplier les cycles d'essais et d'erreurs, amplifiant l'empreinte de chaque projet. Cela motive un intérêt profond pour des méthodes ou des modèles capables de rendre l'entraînement plus prévisible, plus robuste, et en définitive plus efficace.

Dans cette perspective, il devient souhaitable de limiter autant que possible la nécessité de ré-entraîner entièrement les modèles afin de les améliorer ou afin de leur ajouter de nouvelles capacités. L'idéal serait de pouvoir réutiliser et combiner des compétences déjà apprises, de manière à préserver l'information acquise lors des entraînements précédents. Cela donnerait accès à un outil capable de ne jamais gaspiller un entraînement et/ou une compétence. Chaque apprentissage pourrait être stocké quelque part et mis à disposition, rendant inutile l'idée de devoir ré-entraîner un autre modèle sur une connaissance déjà apprise.

Cette propriété est très recherchée en Machine Learning (*Model Merging* [48], *Continual Learning* [45], *Lifelong Learning* [24], ...) et notamment par Victor Boutin, dont les travaux s'inscrivent indirectement dans cette problématique. Il explore une approche *basée sur l'énergie* qui pourrait permettre d'exploiter cette capacité de fusion de modèles.

C'est dans ce contexte que s'inscrit l'objectif principal de ce stage : rendre cette approche plus accessible et exploitable, en s'appuyant sur les recherches menées au sein de l'équipe DEEL.

2 Introduction et Travaux connexes

Ce formalisme, dit *basé sur l'énergie*, trouve ses racines dans la physique statistique, s'appuyant sur la notion que les configurations les plus probables d'un système sont celles qui minimisent son énergie totale (ex. l'entropie).

L'idée de cette approche est de se servir de cette énergie pour évaluer la probabilité qu'un échantillon appartienne à notre distribution cible. On appelle *distribution cible* la distribution de probabilité dont sont issues les données observées (ex. les images réelles, les exemples d'entraînement), et que l'on cherche à approximer.

La distribution de Boltzmann (ou distribution de Gibbs) est pionnière dans l'utilisation de cette approche "énergétique" pour modéliser la probabilité d'une donnée x d'appartenir à une distribution :

$$p(x) = \frac{e^{-E(x)}}{Z} \tag{1}$$

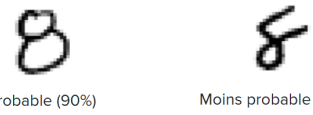
Où :

- $E(x)$ représente l' **énergie** associée à la donnée x. Dans ce paradigme, plus l'énergie $E(x)$ est faible, plus la donnée x est considérée comme probable (ou "proche" de la distribution de données réelle).
- Z est la **fonction de partition**, une constante de normalisation qui assure que la distribution est bien une densité de probabilité (Son intégrale sur l'ensemble des données vaut 1) :

$$Z = \int e^{-E(x)} dx \tag{2}$$

Considérons le dataset, MNIST (jeu de données de caractères manuscrits, ci-contre), l'objectif serait de pouvoir déterminer à partir d'une image si cette dernière suit la même distribution que MNIST :





Dans le cas ci-dessus, le but serait d'obtenir une énergie faible pour l'image de gauche et une plus forte énergie pour l'image de droite.

Cette approche "énergétique" peut être comparée à la façon dont notre cerveau évalue la familiarité d'une situation. Imaginez que vous croisez une personne dans la rue : si son visage vous semble familier (comme le 8 à gauche), votre cerveau associe une sorte de "sentiment de faible énergie", une impression de déjà-vu, comme si cette information "colle" bien avec vos souvenirs (à votre distribution).

À l'inverse, si la personne a un visage inconnu ou qui ne ressemble pas à quelqu'un que vous connaissez (comme le faux 8 à droite), votre cerveau génère une "énergie" plus élevée, un signal d'alerte indiquant que quelque chose ne correspond pas à quelque chose que vous avez déjà vu (qui n'est pas dans votre distribution) [32, 31].

De la même manière, comme notre cerveau peut nous tromper avec des fausses impressions de déjà-vu, cette approche peut parfois attribuer une énergie trop basse à une anomalie, ou trop haute à un échantillon valide, créant une illusion de familiarité (ou d'étrangeté) erronée [11].

Cette analogie illustre aussi pourquoi ces modèles sont puissants pour des tâches comme la détection d'anomalies : ils capturent cette capacité humaine à discriminer intuitivement ce qui est statistiquement attendu de ce qui dévie des patterns habituels, sans qu'une définition explicite de l'anomalie soit nécessaire. [53]

Notre objectif devient donc d'attribuer une énergie à chaque instance, ramenant notre cas MNIST non pas à une recherche de probabilité, mais à une étude énergétique de l'image (voir Figure 3) :



FIGURE 3 – Exemple de l'approche énergétique sur MNIST

Le paysage énergétique représenté par cette approche (ici projetée en 2D pour simplifier, voir Figure 4) reflète donc exactement l'objectif illustré ci-dessus. Les images "typiques" de MNIST (comme le chiffre bien formé) se retrouvent dans la zone de basse énergie, tandis que les anomalies ou images non représentatives (comme le bruit ou un chiffre déformé) sont repoussées vers la zone de haute énergie.

La forme globale de la courbe matérialise ainsi ce que le modèle reconnaît comme "familier" et ce qu'il identifie comme "étranger", une approche énergétique, et non probabiliste .

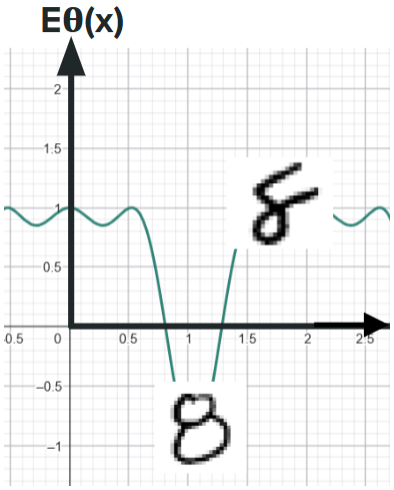


FIGURE 4 – Paysage énergétique type de cette approche

Un aspect fondamental de cette formulation est qu'elle introduit naturellement des propriétés de **composition**. Lorsque plusieurs modules apprennent chacun une fonction d'énergie $E_i(x)$ capturant une contrainte particulière sur une variable x et que ces variables sont mutuellement

indépendantes, il devient possible de les combiner au moyen d'un opérateur de composition adapté. Par exemple, sous hypothèse d'indépendance, il est possible de combiner les contraintes en sommant les énergies entre elles [16, 17] :

$$E_{total}(x) = \sum_i E_i(x) \tag{3}$$

Ainsi, la probabilité d'appartenance à la distribution du modèle est définie par :

$$p_{final}(\mathbf{x}) \propto e^{-(E_1(\mathbf{x})+E_2(\mathbf{x}))} * \frac{1}{Z} \tag{4}$$

Où Z, la fonction de partition (Eq. 2) doit être calculée post-entraînement.

Exemple

Un modèle reconnaît les chiens (E_1), un autre les chats (E_2).

1. Soustraire leurs énergies permet d'obtenir une probabilité de Boltzmann qui fusionne leurs connaissances sans ré-entraînement ni post-traitement :

$$p_{final}(\mathbf{x}) \propto \frac{e^{-(E_1(\mathbf{x})-E_2(\mathbf{x}))}}{Z} = \frac{p_1(\mathbf{x})}{p_2(\mathbf{x})} * \frac{1}{Z}$$

2. Il en ressort une décision unifiée : *À quel point l'image est-elle plus chien que chat ?*
   - $p_{final} \to 0$: chat
   - $p_{final} \to 1$: chien
   - $p_{final} \to 0.5$: indéterminé

Tout ça sans avoir à ré-entraîner un nouveau modèle. Cela revient à résoudre un problème en tenant compte de l'ensemble des contraintes simultanément, plutôt que d'essayer de traiter chaque contrainte séparément et de fusionner ensuite les résultats (comme avec plusieurs classifieurs) [30].

Cette composabilité est une conséquence de la nature additive des log-probabilités, et est propre au cadre de cette approche.

Bien avant l'essor du deep learning, Zhu et al. [56] ont proposé le modèle **FRAME** (Filters, Random fields And Maximum Entropy). Conçu pour modéliser des textures naturelles, il repose sur l'utilisation de filtres fixes (faits main) qui sont l'équivalent de fonctions d'énergie.

Chaque filtre contient une connaissance/compétence et l'ensemble des filtres (des énergies) calculés permet une décision finale intelligente sur la probabilité (au sens de Boltzmann) d'appartenance à une distribution cible (ici de l'herbe, des feuilles d'arbres, etc...) :

$$p(I) = \frac{e^{-\sum_k \lambda_k f_k(I)}}{Z} \tag{5}$$

Dans cette formulation :

- I représente l'image.
- $f_k(I)$ est la réponse de l'image à un filtre prédéfini (par exemple, Difference of Gaussians).
- λ_k sont les poids appris par maximum de vraisemblance pour ajuster chaque contrainte.

Ces travaux fondateurs ont posé le cadre théorique sur lequel s'appuient les recherches menées par Victor Boutin, qui mobilisent aujourd'hui le *deep learning* à travers des **Energy-Based Models** : des modèles plus flexibles, capables d'inférer directement les fonctions d'énergie à partir des données, sans ingénierie manuelle des filtres ni connaissance a priori de la structure sous-jacente.

2.1 Modèles Basés sur l'Énergie (EBM)

Après l'avènement du Machine Learning, de nombreux projets de recherche ont proposé des solutions pour modéliser ces fonctions d'énergie. Ackley et al. [41], ainsi que Salakhutdinov et Hinton [39, 18], ont notamment introduit des modèles basés sur l'énergie utilisant des variables latentes, où l'énergie est exprimée comme une combinaison de variables latentes et observables.

LeCun et al. [25] ont proposé un formalisme général des Modèles Basés sur l'Énergie (EBM, de l'anglais *Energy-Based Model*). Une contribution majeure fut cependant celle de Du et al. [10], qui a clarifié et formalisé l'approche consistant à modéliser la fonction d'énergie au moyen d'un réseau de neurones, tout en démontrant le potentiel génératif des EBMs. Ce modèle définit une distribution de probabilité sur les données $\mathbf{x} \in \mathcal{X}$ à travers une *fonction d'énergie* scalaire $E_\theta : \mathcal{X} \to \mathbb{R}$. La probabilité de Boltzmann attribuée à un point de données devient donc :

$$p_\theta(\mathbf{x}) = \frac{e^{-E_\theta(\mathbf{x})}}{Z(\theta)}, \qquad Z(\theta) = \int_{\mathcal{X}} e^{-E_\theta(\mathbf{x})}\, d\mathbf{x} \tag{6}$$

où $Z(\theta)$ est la *fonction de partition* qui dépend ainsi des paramètres θ du modèle.

Le défi principal est que $Z(\theta)$ est intractable pour tout E_θ non trivial, et cela nous oblige à recourir à des approximations. En effet, cela implique de pouvoir intégrer sur tout l'espace des données x, qui sont de très haute dimension.

Idée clé

Nous souhaitons utiliser le machine learning pour apprendre notre fonction d'énergie. Cependant, les réseaux de neurones sont des fonctions complexes et peu interprétables. Cette approche nous évite donc de devoir déterminer manuellement une fonction d'énergie, mais nous fait hériter des propriétés intrinsèques aux réseaux de neurones.

La difficulté réside alors dans le fait que nous n'observons jamais l'intégralité du paysage énergétique : seuls des échantillons nous sont accessibles.

2.1.1 Entraîner un EBM : Maximum de vraisemblance et divergence contrastive

Entraîner un EBM revient à imposer au réseau de neurones une faible énergie sur notre jeu de données (ou distribution) et une forte énergie dans les autres cas. Nous sommes donc dans une situation d'apprentissage non supervisé, qui nous dispense du besoin de labelliser nos données, etc...

Nous souhaitons, plus précisément, que $p_\theta(x)$ approxime la distribution réelle des données $p_D(x)$; Nous cherchons donc à minimiser une Decoupled Kullback-Leibler Divergence Loss [9] :

$$DKL(p_D \| p_\theta) = \mathbb{E}_{x \sim p_D}\left[\log \frac{p_D(x)}{p_\theta(x)}\right] = \mathbb{E}_{x \sim p_D}\left[\log p_D(x) - \log p_\theta(x)\right]$$

La DKL est une mesure de proximité entre deux distributions. L'objectif est de minimiser cette distance pour rapprocher p_θ de p_D. Posons donc le problème d'optimisation suivant :

$$\min_\theta -\mathbb{E}_{x\sim p_D}[\log p_D(x) - \log p_\theta(x)]$$

Or $p_D(x)$ est fixe donc :

$$\min_\theta \mathbb{E}_{x\sim p_D}[\log p_\theta(x)]$$

En remplaçant p_θ avec (6), il est possible d'écrire la fonction de coût $\mathcal{L}$ d'un EBM comme :

$$\mathcal{L}(\theta) = \mathbb{E}_{x\sim p_D}[E_\theta(x)] + \log Z(\theta) \tag{7}$$

Or comme dit ci-dessus $\log Z(\theta)$ est intractable pour un réseau de neurones E_θ. Cependant Turner propose une solution à ce problème [43] :

$$Z(\theta) = \int e^{-E_\theta(x)} dx \qquad \textbf{alors} \qquad \nabla_\theta Z(\theta) = \int e^{-E_\theta(x)} \left(-\nabla_\theta E_\theta(x)\right)\, dx \tag{8}$$

De plus,

$$\nabla_\theta \log Z(\theta) = \frac{1}{Z(\theta)} \nabla_\theta Z(\theta) \tag{9}$$

Donc, avec (8) et (9),

$$\nabla_\theta \log Z(\theta) = \frac{1}{Z(\theta)} \int e^{-E_\theta(x)} \left(-\nabla_\theta E_\theta(x)\right)\, dx$$

Et, sachant $Z(\theta)$ constante selon x,

$$\nabla_\theta \log Z(\theta) = \int \frac{e^{-E_\theta(x)}}{Z(\theta)} \left(-\nabla_\theta E_\theta(x)\right)\, dx \tag{10}$$

En utilisant la définition de $p_\theta(\mathbf{x})$ dans (6) avec la partie rouge dans (10) :

$$\nabla_\theta \log Z(\theta) = -\int p_\theta(x)\, \nabla_\theta E_\theta(x)\, dx$$

Cette intégrale représente la moyenne des gradients de l'énergie sous la distribution p_θ. Posons $\mathbf{x}^-$ les éléments de la distribution de p_θ, ce qui nous permet de réécrire l'expression comme suit :

$$\nabla_\theta \log Z(\theta) = -\mathbb{E}_{x^-\sim p_\theta}[\nabla_\theta E_\theta(x^-)] \tag{11}$$

Or, le gradient de (7) est :

$$\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_{x\sim p_D}[\nabla_\theta E_\theta(x)] + \nabla_\theta \log Z(\theta) \tag{12}$$

Nous obtenons finalement en remplaçant avec (11) dans (12), le gradient de la fonction de coût $\mathcal{L}$.

$$\nabla_\theta \mathcal{L}(\theta) = \mathbb{E}_{x^+\sim p_D}[\nabla_\theta E_\theta(x^+)] - \mathbb{E}_{x^-\sim p_\theta}[\nabla_\theta E_\theta(x^-)] \tag{13}$$

Et en intégrant :

$$\mathcal{L}(\theta) = \mathbb{E}_{x^+\sim p_D}[E_\theta(x^+)] - \mathbb{E}_{x^-\sim p_\theta}[E_\theta(x^-)] \tag{14}$$

Ce formalisme a une interprétation claire : la descente de gradient selon (13) aura pour objectif d'*abaisser* l'énergie des échantillons réels $\mathbf{x}^+$ et de *relever* l'énergie des échantillons du modèle $\mathbf{x}^-$.

Ainsi, le problème à résoudre est maintenant d'échantillonner des éléments $\mathbf{x}^-$ de la distribution p_θ. La littérature propose plusieurs solutions à ce problème, mais dont les deux principaux sont :

1. **Génération Amortie** : inspirée des **GANs** ou des **VAE**, cette approche utilise un *générateur* à part (exemple : réseau de neurones) pour produire des échantillons $\mathbf{x}^-$ [28, 46]. Cette méthode nécessite donc un entraînement conjoint du générateur et du modèle d'énergie, ce qui peut introduire des biais. Plusieurs méthodes de génération ont été explorées, comme les modèles de diffusion [50], les auto-encodeurs variationnels [47], ou d'autres méthodes utilisant un réseau distinct pour générer des échantillons [14, 22, 54, 16, 23], ce qui rend ces méthodes étroitement liées aux GANs [12].
2. **MCMC (Markov Chain Monte Carlo) à pas de Langevin :** utilise des **chaînes de Markov** pour échantillonner p_θ en exploitant le gradient de l'EBM. Hinton [19] initialise ces chaînes sur des données réelles (pas "informés"), ce qui permet un apprentissage contrastif efficace. Cette approche reste cependant limitée : les chaînes, ne faisant que quelques pas autour des données, n'explorent jamais les régions éloignées (comme le bruit). Tieleman [42] propose la *Persistent Contrastive Divergence*, qui ne réinitialise plus les chaînes à chaque itération, mais qui les propage tout au long de l'apprentissage, ce qui aide à l'exploration du réseau. Nous baserons nos travaux sur une méthode introduite par Du et al. [10], qui règle ce problème en permettant la génération des échantillons de p_θ en partant du bruit. Cette méthode est décrite dans la section suivante.

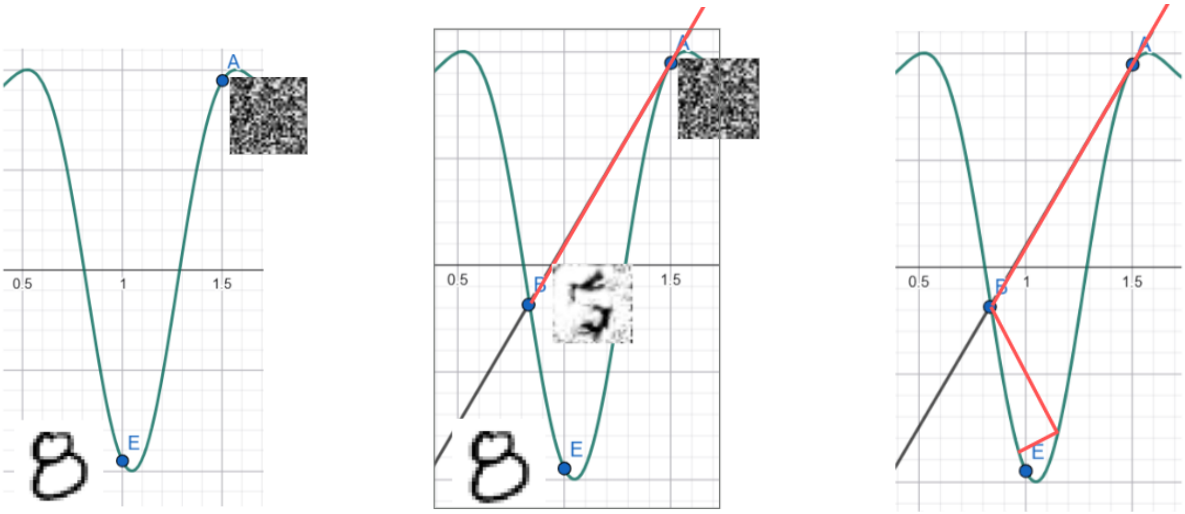
2.2 Échantillonnage MCMC par dynamique de Langevin

2.2.1 Dynamique de Langevin non-ajustée (ULA, Unadjusted Langevin Algorithm)

Nous souhaitons tirer des échantillons de $p_\theta \propto e^{-E_\theta}$ à l'aide de la dynamique de Langevin. Apparue pour la première fois dans les travaux de G. Parisi [36], la dynamique de Langevin repose sur des itérations stochastiques à partir d'un point initial $\mathbf{x}_0$:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\epsilon}{2}\nabla_{\mathbf{x_t}} E_\theta(\mathbf{x}_t) + \sqrt{\epsilon}\,\boldsymbol{\eta}_t, \qquad \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{15}$$

où $\epsilon > 0$ est la taille du pas. Lorsque $\epsilon \to 0$ et que le nombre de pas $t \to \infty$, la distribution de $\mathbf{x}_t$ converge vers p_θ. En pratique, ϵ et t sont finis, ce qui introduit un biais.



(a) État initial $\mathbf{x}_0$ (b) 1 pas de Langevin (c) λ pas de Langevin

FIGURE 5 – Étapes successives d'une itération de Langevin. Le gradient guide la chaîne vers les régions de basse énergie

Remarque 1. *La mise à jour de l'équation (15) déplace l'échantillon courant dans la direction du gradient négatif de l'énergie (vers les régions de basse énergie, voir Figure 5), tandis que le terme de bruit $\sqrt{\epsilon}\,\boldsymbol{\eta}_t$ permet à l'exploration d'être expressive et variée, ce qui empêche la chaîne de se concentrer sur un seul mode.*

Il est primordial de comprendre que la dynamique de Langevin a pour objectif d'échantillonner notre distribution apprise p_θ et non notre distribution cible d'apprentissage (ex. MNIST). Si notre échantillonnage est mauvais, alors la fonction de coût (14) ne permet pas un apprentissage correct. Il faut donc s'assurer que notre chaîne de Markov se dirige correctement vers la distribution de notre EBM p_θ, de manière stable et stationnaire (rajouter un pas ou cent pas une fois la distribution atteinte ne doit surtout pas nous en écarter). De plus, un pas de Langevin est très coûteux en raison du calcul de gradient qu'il nécessite par itération ; il faut donc réduire au maximum le nombre de pas utilisés.

2.2.2 MCMC à courte durée

Dans la littérature sur l'entraînement des EBM, un choix pratique courant est d'utiliser des *MCMC à courte durée* : au lieu de faire tourner les chaînes jusqu'à convergence, est effectué uniquement λ pas (typiquement $\lambda \in [20, 100]$) de Langevin à partir d'une distribution fixe (généralement du bruit). Ce choix est plus raisonnable en termes de calculs, mais les échantillons obtenus n'ont pas complètement convergé vers la distribution p_θ.

L'observation clé de Nijkamp et al.[33, 34] est que le choix de λ, du pas ϵ, de la distribution d'initialisation et de l'optimiseur a un fort impact sur la dynamique d'entraînement. En particulier, des chaînes mal calibrées peuvent produire des échantillons *dégénérés* qui ne représentent pas du tout la distribution p_θ, ce qui entraîne des échecs d'entraînement.

2.2.3 Dynamique de Langevin ajustée (MALA, Metropolis-Adjusted Langevin Algorithm)

La mise à jour de l'équation (15) correspond à la version non ajustée de la dynamique de Langevin : le terme « non ajusté » vient de l'absence d'une étape de correction de Metropolis-Hastings (MH) qui, dans la version ajustée (MALA, Metropolis-Adjusted Langevin Algorithm [35]), accepte ou rejette chaque proposition x_{t+1} selon un ratio d'acceptation garantissant que la chaîne a bien p_θ comme distribution stationnaire exacte.

Sans cette correction, la dynamique de Langevin avec pour pas ϵ introduit un biais systématique : la distribution stationnaire de la chaîne n'est plus exactement p_θ mais une distribution légèrement différente dont l'écart à p_θ croît avec ϵ.

Idée clé

En effet, si le Langevin se déplace continuellement et avec un ϵ fixe, il risque de ne pas converger complètement vers la distribution. (ex. Figure 5c)

Cette correction garantit que la chaîne n'est pas biaisée. En contrepartie, chaque pas nécessite deux évaluations de E_θ (pour la proposition et pour le ratio), ce qui double le coût computationnel par rapport à la version non-ajustée. De plus, l'étape d'acceptation-rejet introduit une variance supplémentaire dans les gradients, ce qui peut déstabiliser l'entraînement lorsque ϵ est trop grand. En pratique, la dynamique de Langevin non-ajustée est préférée pour l'entraînement des EBMs, car la version ajustée est trop gourmande en termes de calcul et d'autres méthodes permettent de s'assurer que le Langevin se passe bien, comme le **buffer replay.**

2.2.4 Buffer Replay

Mathématiquement, faire de l'échantillonnage MCMC par dynamique de Langevin avec un nombre de pas infini résulte forcément en un apprentissage stabilisé et convergent. Afin d'imiter ce comportement, Du et al. [10] proposent le mécanisme de *buffer replay* : plutôt que d'initialiser chaque chaîne de Markov à partir du bruit $\tilde{x}_0 \sim \mathcal{N}(0, 1)$ (la forme que peut prendre ce bruit (uniforme, gaussien, etc...) est discutée en Annexe E), ils proposent de maintenir un ensemble $\mathcal{B}$ de N échantillons persistants issus des itérations précédentes d'entraînement.

Procédure. À chaque étape d'entraînement :

1. On tire $\tilde{x}_0$ uniformément dans $\mathcal{B}$
2. Avec probabilité r (typiquement 0.05), on ré-initialise $\tilde{x}_0$ depuis du bruit uniforme
3. On effectue λ pas de Langevin à partir de $\tilde{x}_0$ pour obtenir x_λ^-
4. On met à jour le buffer : $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{\tilde{x}_0\}) \cup \{x_\lambda^-\}$
5. On utilise x_λ^- comme échantillon négatif pour la fonction de coût (14)

L'intuition centrale est qu'après plusieurs itérations (et donc plusieurs affinages du buffer), les échantillons stockés approximent une chaîne plus longue et donc de meilleure qualité. Ainsi, les échantillons générés par l'EBM voient, au fil de l'entraînement, beaucoup plus de pas de plus en plus précis. Combinée aux pas de Langevin, cette méthode permet de stabiliser l'apprentissage des EBMs et servira de base pour ce travail. Cependant, la mécanique de Langevin impose, même avec cette approche, des **limitations et des instabilités**.

2.3 Limitations

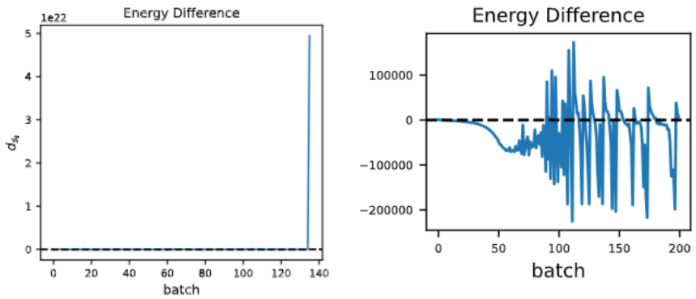
2.3.1 Instabilité numérique

Nijkamp et al. [33] ont mis en évidence plusieurs limitations et instabilités propres aux EBM. Ils ont par ailleurs démontré que cette instabilité dépend fortement des hyper-paramètres choisis, pouvant mener à plusieurs cas distincts :

— **L'expansion** : qui implique une explosion des gradients. Les mises à jour deviennent trop amples et la chaîne diverge (voir Figure 6a).

— **La contraction** : qui implique une évanescence des gradients. L'EBM n'est pas capable de différencier la distribution générée par le Langevin de la vraie distribution.

— **L'oscillation** : la fonction de coût oscille entre des énergies élevées et basses, entre expansion et contraction, ce qui prouve la capacité des pas de Langevin à se rapprocher de la distribution et celle de l'EBM à quand même les discerner (voir Figure 6b).



(a) Explosion (b) Oscillation

FIGURE 6 – Comportements instables de la fonction de coût d'un EBM

Plusieurs méthodes ont été développées pour stabiliser l'apprentissage et prévenir les problèmes d'instabilité numérique tels qu'une explosion des gradients. Une approche notable, proposée par Yin et al. [49], consiste à modifier la fonction de coût afin d'introduire une régulation automatique :

$$\mathcal{L}_{stab} = \mathbb{E}_{x^+ \sim p_D}[-\alpha(x^+)E_\theta(x^+)] - \mathbb{E}_{x^- \sim p_\theta}[-\beta(x^-)E_\theta(x^-)] \tag{16}$$

où les coefficients, considérés comme constants pour le calcul du gradient, sont définis par :

— $\alpha(x) = 1 - \frac{1}{1+e^{-E_\theta(x)}}$ pour les données réelles

— $\beta(x) = \frac{1}{1+e^{-E_\theta(x)}}$ pour les échantillons négatifs

Cette pondération agit comme un mécanisme de régularisation intrinsèque : lorsque $-E_\theta(x)$ devient trop élevé pour les échantillons de p_D, le facteur $\alpha(x)$ tend vers zéro, atténuant ainsi la contribution du gradient et prévenant l'instabilité numérique. Pareillement, si $-E_\theta(x)$ devient excessivement faible pour les échantillons générés, $\beta(x)$ s'approche de zéro. Bien que cette formulation stabilise l'entraînement, elle tend à limiter l'apprentissage au support de p_D plutôt qu'à la densité complète.

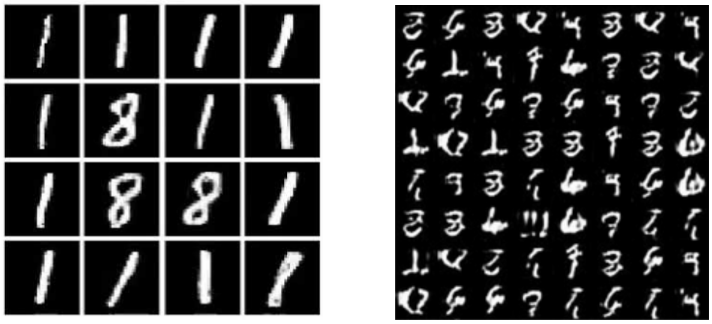
Les **Joint Energy-Based Models (JEBM)** [15] constituent une autre approche notable dans ce domaine. Plutôt que de modéliser la distribution marginale $p(x)$, ces modèles apprennent la distribution jointe $p(x, y)$, liant ainsi les images à leurs catégories et utilisant la classification pour guider la forme du paysage énergétique. Cette approche offre de meilleures performances en classification, mais elle reste tout aussi instable à entraîner qu'un EBM classique, tout en s'avérant plus coûteuse en calcul et en nécessitant impérativement des données étiquetées. Ces méthodes conjointes sortent donc du cadre de ce travail.

2.3.2 Problèmes de convergence

Un EBM peut *sembler* apprendre (fonction de coût qui oscille de manière contrôlée), mais ne pas converger complètement vers la distribution cible. Nijkamp et al. [33] présentent deux scénarios critiques :

— **Mode collapse** : la chaîne de Langevin se concentre sur un sous-ensemble de modes, ignorant les autres régions de l'espace. Cela se manifeste par une fonction de coût qui *semble* converger, mais qui masquerait une exploration incomplète. (voir Figure 7a)

— **Mauvais hyper-paramètres** : un mauvais paramétrage peut ralentir et empêcher la distribution générée de se rapprocher suffisamment de la distribution cible. (voir Figure 7b)



(a) p_θ ne représente qu'une partie de p_D

(b) p_θ n'a pas complètement convergé vers p_D

FIGURE 7 – Exemple de mode collapse (gauche) et non convergence (droite) d'un EBM

2.3.3 Trajet Bruit → Distribution : Le défi de la haute dimension

Même avec un replay buffer, l'apprentissage d'un EBM sur des données complexes peut être délicat. Bien que la dynamique de Langevin converge théoriquement vers p_θ indépendamment de son point d'initialisation, le nombre limité d'itérations utilisées peut empêcher les chaînes d'atteindre p_D. Cette mauvaise exploration produit des échantillons négatifs de faible qualité, ce qui peut entraîner des instabilités d'entraînement, voire un *mode collapse*.

Idée clé

L'EBM est performant si le gradient utilisé dans le Langevin permet de converger rapidement vers la distribution. Plus cette tâche est facile, plus l'apprentissage est stable.

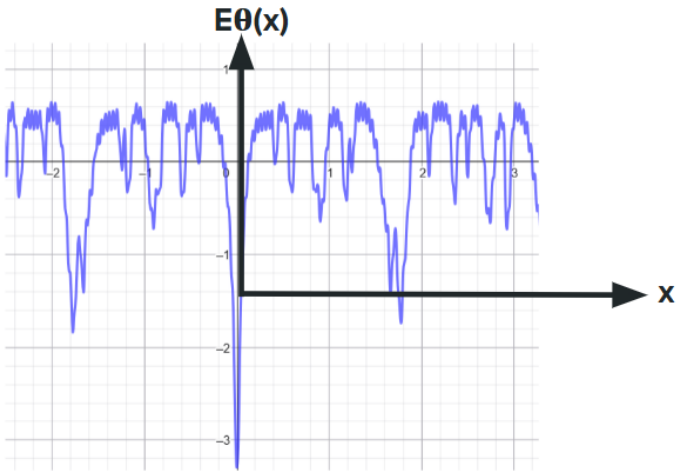
Pour essayer de pallier ce problème, Balcerak et al. proposent d'unifier les EBMs avec du **Flow Matching** [3]. Le Flow Matching résout un problème de *transport*, où il apprend une trajectoire paramétrée et connue entre une distribution source et une distribution cible. Cette trajectoire fournit un point de départ x_0 déjà proche de p_D pour le Langevin, ce qui réduit le nombre d'itérations nécessaires pour que la chaîne atteigne une région de haute densité sous p_θ.

Zhao et al. proposent une autre approche, un apprentissage multi-échelle par **Downsampling**, afin d'éviter que l'EBM ne se perde dans les détails de haute fréquence (bruit local) dès le début de l'apprentissage. Leur stratégie consiste à opérer par *downsampling* progressif [55]. Cette hiérarchie de résolution permet de construire un pont entre le bruit et la distribution complexe, garantissant que les caractéristiques structurelles majeures sont apprises avant que le modèle ne tente de modéliser les détails les plus fins.

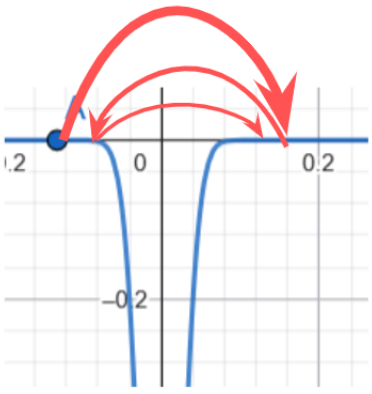
Ces travaux démontrent l'importance cruciale de l'échantillonnage et du point de départ de la chaîne de Langevin dans la stabilité de l'apprentissage d'un EBM, et motiveront par la suite les travaux présentés dans ce rapport.

2.3.4 Paysage énergétique

Un EBM bien entraîné apprend un **paysage énergétique** qui reflète notre distribution cible avec des **pics abrupts** pour les échantillons positifs (basse énergie, voir Figure 8a).



(a) Schématisation d'un paysage énergétique d'un EBM *sharp* : des pics abrupts et des minimums locaux.



(b) Étapes de Langevin près d'un pic : la chaîne peut osciller longtemps avant de "tomber" dans un puits (basse énergie).

FIGURE 8 – Exemple typique de l'apprentissage d'un EBM

Cette *sharpness* peut piéger le Langevin dans des minima locaux (Figure 8a) ou lui faire manquer les puits d'énergie (Figure 8b), compliquant ainsi la tâche du gradient $\nabla_{\mathbf{x}} E_\theta(\mathbf{x})$. **C'est ici qu'interviennent les travaux de l'équipe DEEL**, partant de l'hypothèse que la sharpness du paysage énergétique, caractérisée par des pics prononcés et des variations locales abruptes, constitue un facteur d'instabilité de l'entraînement des EBMs, et que son lissage en est un levier de correction. Cette hypothèse motive plusieurs méthodes de "lissage" reconnues, étudiées par DEEL, comme **la contrainte de Lipschitz**, décrite dans la section suivante.

2.4 La contrainte de Lipschitz

La contrainte de Lipschitz constitue un candidat naturel à ce problème, en imposant une borne explicite sur la variation de E_θ. Cette contrainte borne la variation maximale que peut subir la sortie d'une fonction pour un déplacement donné dans son domaine, empêchant ainsi toute variation brusque du gradient. Formellement :

Définition 1 (Continuité Lipschitz)**.** *Une fonction $f : \mathbb{R}^n \to \mathbb{R}^m$ est dite K-Lipschitz (par rapport à la norme euclidienne) s'il existe une constante $K > 0$ telle que pour tout $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:*

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\| \tag{17}$$

*Est appelée **constante de Lipschitz** de f la plus petite valeur de K satisfaisant cette condition.*

En tout point $\mathbf{x}$ où une fonction K-Lipschitz f est différentiable, alors $\|\nabla_{\mathbf{x}} f(\mathbf{x})\| \leq K$ presque partout. De plus cela implique que pour un écart en norme de 1 entre $\mathbf{x}$ et $\mathbf{y}$, il ne peut y avoir au maximum qu'un écart de $\mathbf{K}$ entre $f(\mathbf{x})$ et $f(\mathbf{y})$.

Ainsi, nous obtenons un hyper-paramètre supplémentaire qui nous permet de décider la *smoothness* du paysage énergétique de E_θ (cf. Figure 9)

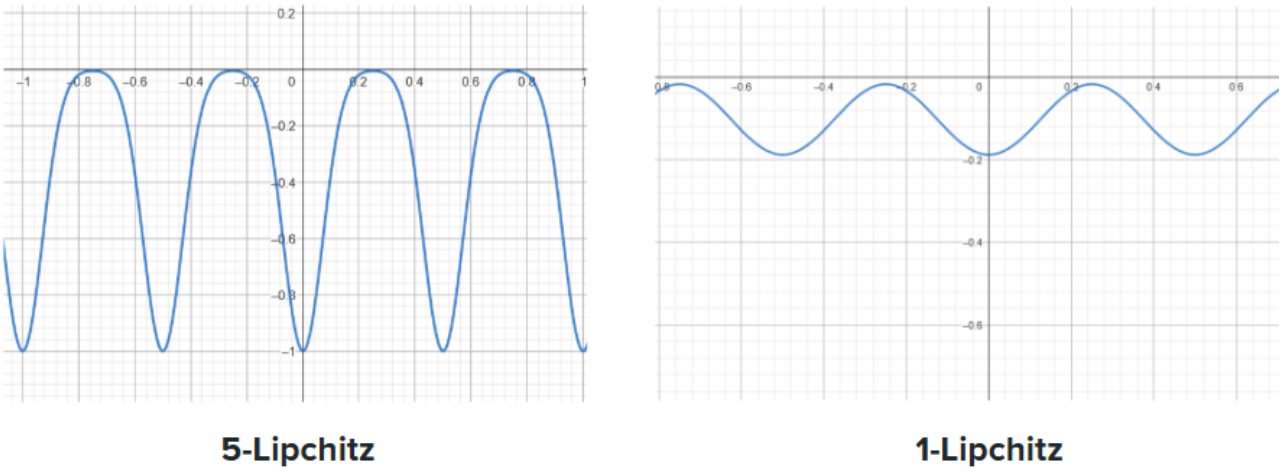


FIGURE 9 – Exemple d'un paysage énergétique K-Lipschitz

Ici, **plus K est petit**, plus le paysage est ***smooth***. De plus, Béthune et al. [8] ont prouvé qu'en théorie, cette contrainte n'a aucun impact sur l'expressivité sur des tâches de classification. En revanche, pour des problèmes de régression, une fonction K-Lipschitz ne peut approximer correctement qu'une fonction cible elle-même K-Lipschitz, ce qui peut donc possiblement limiter sa capacité à modéliser un paysage énergétique arbitrairement *sharp* et complexe tout en respectant un écart maximal de K entre $\mathbf{x}$ et $\mathbf{x} \pm 1$.

Imposer cette contrainte est une technique reconnue, notamment pour la robustesse adversariale [7, 29], mais elle reste peu utilisée en pratique en raison de la limite d'expressivité qu'elle impose et de la complexité de contraindre un modèle à être strictement Lipschitz. Dans le cadre des EBMs, il est admis que limiter la constante de Lipschitz aide à stabiliser l'apprentissage [33] ; cependant, à notre connaissance, aucun travail n'a encore imposé cette contrainte au sens strict pour des EBMs. Pour cela, nous utilisons `deel-torchlip`, une bibliothèque PyTorch open-source développée par l'équipe **DEEL à l'IRT Saint Exupéry** [4], qui permet de construire des réseaux strictement Lipschitz selon les méthodes présentées dans la section suivante.

2.5 Réseaux de neurones et contrainte de Lipschitz

2.5.1 Principe général

Calculer exactement la constante de Lipschitz d'un réseau de neurones est un problème NP-difficile, y compris pour des architectures simples telles que les perceptrons multicouches. En pratique, il est donc nécessaire de se contenter d'une **borne supérieure** sur cette constante, obtenue en contraignant la constante de Lipschitz de chaque couche du réseau. [1, 40] Ainsi, pour un réseau de neurones composé de L couches, noté $f = f_L \circ \cdots \circ f_1$, la constante de Lipschitz globale satisfait l'inégalité suivante :

$$\mathrm{Lip}(f) \leq \prod_{l=1}^{L} \mathrm{Lip}(f_l) \tag{18}$$

Pour construire un réseau K-Lipschitz, il suffit de s'assurer que chaque couche f_l est K_l-Lipschitz avec $\prod_l K_l = K$. Dans le cas le plus simple, si chaque couche est contrainte à être 1-Lipschitz, alors la borne globale est 1.

2.5.2 Méthodes pour imposer la contrainte 1-Lipschitz

1. Normalisation spectrale. L'approche la plus simple pour les couches linéaires consiste à diviser la matrice de poids par une estimation de sa norme spectrale avant de l'appliquer à l'entrée x :

$$\hat{W}_l = \frac{W_l}{\hat{\sigma}_{\max}(W_l)} \quad \textbf{avec} \quad f_l(\mathbf{x}) = W_l\mathbf{x} + \mathbf{b}_l \quad \textbf{et} \quad \mathrm{Lip}(f_l) = \|W_l\|_2 = \sigma_{\max}(W_l) \tag{19}$$

où $\sigma_{\max}(W_l)$ est la plus grande valeur singulière de W_l (la norme spectrale) et est estimée efficacement par itération de puissance. Imposer $\sigma_{\max}(W_l) \leq 1$ permet donc d'obtenir une couche linéaire 1-Lipschitz. C'est l'approche la plus utilisée aujourd'hui [1].

2. Orthogonalisation de Björck. Anil et al. ont montré que pour des tâches de classification, il pouvait être intéressant de projeter la matrice de poids sur la variété des matrices orthogonales (dont toutes les valeurs singulières sont égales à 1). L'algorithme de Björck & Bowie [6] y parvient de manière itérative. En partant d'une matrice de poids $W^{(0)}$ normalisée spectralement, l'itération est :

$$W^{(k+1)} = W^{(k)} \left(\mathbf{I} + \frac{1}{2} \left(\mathbf{I} - {W^{(k)}}^{\top} W^{(k)} \right) \right) \tag{20}$$

qui converge vers la matrice orthogonale la plus proche de $W^{(0)}$ (en norme de Frobenius). En pratique, un petit nombre d'itérations (ex. $n_{\text{iter}} \in [5, 15]$) suffit.

Le pipeline que nous utiliserons combine la normalisation spectrale suivie de l'orthogonalisation de Björck. Cette procédure en deux étapes amène d'abord la norme spectrale proche de 1, puis affine les poids pour qu'ils soient exactement orthogonaux. Puisque les matrices orthogonales satisfont $W^\top W = I$, elles ont toutes leurs valeurs singulières égales à 1 et sont donc exactement 1-Lipschitz par construction.

Algorithm 1 Orthogonalisation de Björck (reparamétrisation des poids)

Require: Matrice de poids $W^{(0)} \in \mathbb{R}^{m \times n}$, nombre d'itérations n_{iter}
Ensure: $\hat{W}$ approximativement orthogonale avec $\sigma_{\max}(\hat{W}) \approx 1$
1: $W \leftarrow W^{(0)} / \hat{\sigma}_{\max}(W^{(0)})$ ▷ Normalisation spectrale initiale
2: **for** $k = 1, \ldots, n_{\text{iter}}$ **do**
3: $\quad W \leftarrow W\left(\mathbf{I} + \frac{1}{2}\left(\mathbf{I} - W^\top W\right)\right)$
4: **end for**
5: **return** W

Remarque sur les fonctions d'activation. Tout ce qui précède s'applique aux couches linéaires. Pour que le réseau global soit 1-Lipschitz, les fonctions d'activation doivent également être 1-Lipschitz. Des choix standards comme ReLU et Tanh sont 1-Lipschitz. Cependant, Anil et al. [1] ont également recommandé d'utiliser `GroupSort2`, une fonction d'activation dédiée qui trie les sorties par paires :

$$\text{GroupSort2}(a, b) = (\min(a, b),\ \max(a, b)) \tag{21}$$

GroupSort2 est non seulement 1-Lipschitz, mais aussi *préservatrice de la norme du gradient* : contrairement à ReLU, elle ne rétrécit pas les gradients, ce qui permet de conserver la norme pendant la backpropagation et ainsi ne pas sous-estimer la constante de Lipschitz du réseau [1].

2.5.3 Extension à un réseau K-Lipschitz

Une fois les couches 1-Lipschitz disponibles, construire un réseau K-Lipschitz peut se faire par exemple en composant les couches 1-Lipschitz et en ajoutant un dernier layer qui introduit un facteur K sur la sortie.

Nous verrons dans la section 4 que cet hyper-paramètre a une grande influence sur l'applicabilité des contraintes 1-Lipschitz aux EBMs.

3 Etude des EBMs avec contraintes K-Lipschitz

Nous proposons ainsi dans ce travail d'explorer l'utilisation des réseaux K-Lipschitz pour apprendre des EBMs. Et nous émettons plusieurs hypothèses sur l'avantage de cette contrainte sur l'apprentissage des EBMs.

3.1 Hypothèses

3.1.1 Propriétés des EBMs K-Lipschitz

Notre hypothèse principale est qu'imposer une contrainte de Lipschitz sur le réseau d'énergie permet d'améliorer significativement la stabilité de l'entraînement des EBM basés sur une dynamique de Langevin, tout en conservant une expressivité suffisante pour modéliser des distributions complexes tout en réduisant le nombre de pas nécessaires.

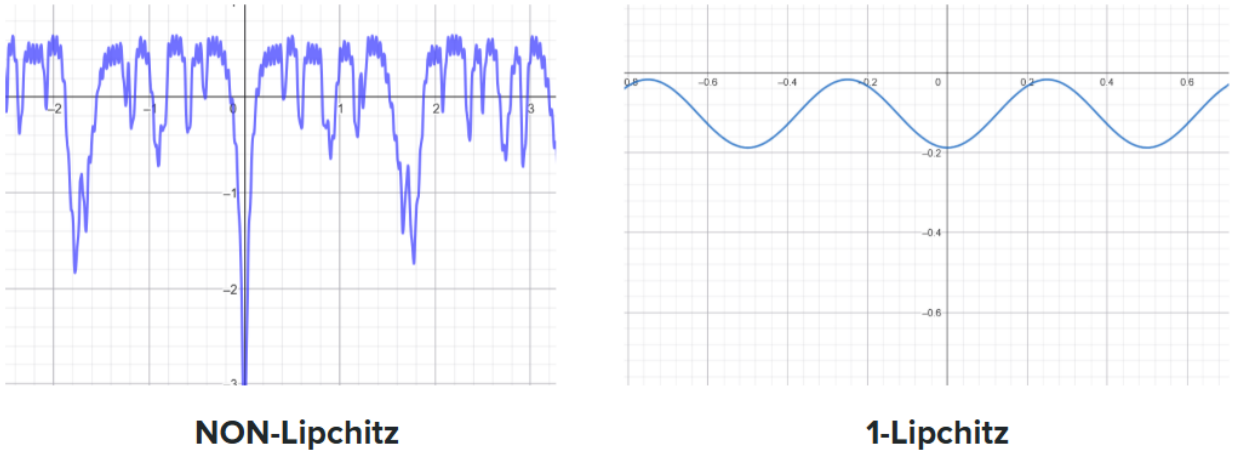


FIGURE 10 – Comparaison du paysage énergétique d'un EBM non contraint et 1-Lipschitz

En effet, nous faisons l'hypothèse qu'un tel paysage énergétique plus régulier facilite directement la dynamique de Langevin utilisée pendant l'entraînement et l'échantillonnage. En particulier :

- les trajectoires MCMC deviennent plus stables ;
- les transitions entre régions de l'espace sont facilitées ;
- les chaînes de Markov ont moins tendance à diverger car le gradient utilisé dans les pas de Langevin est borné par K, et donc la dynamique de Langevin ne peut pas exploser.
- les gradients sont mieux contrôlés ;
- les phénomènes d'explosion des gradients deviennent plus rares.

Autrement dit, une contrainte de Lipschitz agit comme une forme de régularisation structurelle du champ de gradient de l'énergie.

3.1.2 Lien avec la distance de Wasserstein

Cette hypothèse possède également une justification liée à la distance de Wasserstein [2, 44]. La distance de Wasserstein est une distance entre distributions, qui mesure le coût minimal nécessaire pour transformer une distribution de probabilité en une autre. Elle s'interprète comme le coût de transport optimal de masse entre deux distributions. Plus précisément, la distance de Wasserstein d'ordre 1 entre deux distributions $\mathbb{P}_r$ et $\mathbb{P}_g$ est définie par :

$$W_1(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y)\sim\gamma} \left[\|x - y\|\right]$$

où $\Pi(\mathbb{P}_r, \mathbb{P}_g)$ désigne l'ensemble des appairages possibles entre les deux distributions. Cette formulation cherche donc le plan de transport minimisant la distance moyenne parcourue par la masse probabiliste.

Contrairement à des divergences comme la KL-divergence, la distance de Wasserstein reste bien définie même lorsque les supports des distributions ne se recouvrent pas, ce qui la rend particulièrement adaptée aux problèmes génératifs.

Kantorovich et Rubinstein ont proposé [44] une forme duale de la distance de Wasserstein-1 dont la formulation est la suivante :

$$W_1(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\|f\|_L \leq 1} \mathbb{E}_{x^+\sim\mathbb{P}_r}[f(x^+)] - \mathbb{E}_{x^-\sim\mathbb{P}_g}[f(x^-)] \tag{22}$$

Cette formulation fait directement apparaître l'espace des fonctions 1-Lipschitz sur lequel nous minimisons la différence des espérances. De plus, cette distance peut être reliée à l'Eq. 14 de la fonction de coût utilisée pour entraîner les EBMs que nous rappelons ci-dessous.

$$\mathcal{L}(\theta) = \mathbb{E}_{x^+ \sim p_{\mathrm{D}}}[E_\theta(x^+)] - \mathbb{E}_{x^- \sim p_\theta}[E_\theta(x^-)]$$

Ainsi, contraindre le réseau d'énergie à être Lipschitz rapproche théoriquement l'entraînement des EBM du calcul de la distance au sens de Wasserstein entre la distribution générée par un EBM et une distribution cible, tandis que l'objectif du Langevin de l'EBM devient donc la réduction de cette même distance. Cela fournit une interprétation plus stable et géométriquement mieux fondée de l'apprentissage "adverse" d'un EBM.

Idée clé

La contrainte de Lipschitz borne la norme du gradient ($\|\nabla f\| \leq K$), limitant ainsi les variations brutales de f : ses sorties ne peuvent varier de plus de K fois la distance entre les entrées. Un EBM Lipschitz apprend donc une géométrie plus régulière de la distribution des données, ce qui permet de stabiliser le trajet du Langevin vers la distribution cible et d'éviter les overflows. Cette contrainte renforce ainsi les fondations théoriques de la fonction de coût des EBMs.

Remarque 2. *Dans le cas où notre modèle ne serait pas 1-Lipschitz, mais K-Lipschitz (c'est-à-dire un modèle 1-Lipschitz dont on multiplie la sortie par K) la fonction de coût de l'EBM n'évalue plus la distance de Wasserstein entre la distribution générée de l'EBM et la distribution cible, mais K fois cette distance (voir preuve 1).*

Cette nouvelle lecture de la fonction de coût nous permet également dans la section suivante, d'interpréter différemment les mécanismes intervenant dans l'entraînement d'un EBM.

3.1.3 Le transport optimal et entraînement des EBM Lipschitz

Illustrons cela avec le buffer de replay (2.2.4), réinterprétable comme une étape intermédiaire dans la chaîne de transport optimal.

3.1.3.1 Le buffer de replay comme opérateur de transport intermédiaire

Comme vu plus tôt, l'entraînement des EBM par dynamique de Langevin requiert un transport d'une masse probabiliste depuis n'importe quelle distribution initiale p_{init} (ex. gaussienne ou uniforme) jusqu'à p_θ, ce qui représente un coût de transport considérable. C'est pourquoi la technique du *buffer de replay* [10] sert à maintenir une mémoire persistante d'échantillons issus des itérations précédentes, que nous raffinons par quelques pas de Langevin à chaque nouvelle itération.

Formellement, notons $\mathcal{B}_t = \{x_1, \ldots, x_N\}$ le contenu du buffer à l'itération t, constituant une approximation de $p_{\theta_{t-1}}$. Du point de vue du transport optimal, le buffer joue le rôle d'une *distribution intermédiaire* dans la chaîne de transport :

$$p_{\text{init}} \longrightarrow \underbrace{\mathcal{B}_t \approx p_{\theta_{t-1}}}_{\text{buffer}} \longrightarrow p_{\theta_t}$$

Plutôt que de transporter la masse depuis le bruit jusqu'à p_{θ_t} en un seul passage (coût $W_1(p_{\text{init}}, p_{\theta_t})$ potentiellement très élevé), le buffer décompose ce transport en deux étapes

successives. Par l'inégalité triangulaire de la distance de Wasserstein :

$$W_1(p_{\text{init}},\, p_{\theta_t}) \;\leq\; W_1\big(p_{\text{init}},\, p_{\theta_{t-1}}\big) + \underbrace{W_1\big(p_{\theta_{t-1}},\, p_{\theta_t}\big)}_{\text{petit si } \theta \text{ varie lentement}}$$

Le second terme est typiquement petit, puisque θ évolue par petits pas de descente de gradient. Le buffer réduit donc drastiquement le coût de transport effectif que les chaînes MCMC doivent couvrir à chaque itération : À chaque pas, les λ pas de Langevin permettent de mettre à jour les éléments du buffer pour prendre en compte la dérive des paramètres

3.1.3.2 Influence de la contrainte Lipschitz sur le buffer

La pertinence du buffer repose sur une hypothèse implicite : les échantillons stockés $\mathcal{B}_t \approx p_{\theta_{t-1}}$ restent de bons points d'initialisation pour p_{θ_t}, c'est-à-dire que la mise à jour $\theta_{t-1} \to \theta_t$ ne déplace pas brusquement les modes de la distribution. C'est ici que la contrainte Lipschitz joue un rôle central. Si le réseau d'énergie est K-Lipschitz en x, le champ de gradient $\nabla_x E_\theta(x)$ est borné par K en tout point. Cela garantit que :

— Pour tout point x du buffer, $E_\theta(x)$ est borné par K fois sa distance au point le plus proche de p_θ : pas d'explosion d'énergie possible, même après une mise à jour de θ ;

— La norme du gradient $\nabla_x E_\theta(x)$ étant elle aussi bornée par K en tout point, les pas de Langevin effectués à partir des échantillons du buffer restent numériquement stables : il n'existe pas de région où le gradient exploserait brutalement et ferait diverger la chaîne.

En d'autres termes, la contrainte Lipschitz est ce qui garantit que les échantillons du buffer ne deviennent pas soudainement des points à énergie arbitrairement élevée : sans elle, une mise à jour agressive de θ peut créer de nouveaux modes très piqués, et l'énergie d'un échantillon du buffer en un point éloigné n'est alors plus bornée par sa distance à la distribution, ce qui peut forcer les chaînes MCMC à repartir de régions à haute énergie et à diverger.

Le compromis expressivité / stabilité. La constante K gouverne donc possiblement directement l'équilibre entre ces deux propriétés :

— K **grand :** le réseau est plus expressif et peut modéliser des distributions très complexes. En contrepartie, les grands gradients déstabilisent les chaînes de Langevin et accélèrent l'obsolescence du buffer, les mises à jour de θ induisent de fortes discontinuités de paysage, augmentant $W_1(p_{\theta_{t-1}}, p_{\theta_t})$.

— K **petit :** le paysage est lisse, les chaînes MCMC n'explosent plus depuis le buffer, et celui-ci reste valide longtemps. Le modèle perd cependant en capacité à capturer des structures fines.

Idée clé

Nous faisons l'hypothèse que le buffer replay et la contrainte Lipschitz sont deux mécanismes complémentaires :

— Le buffer réduit le coût de transport effectif entre itérations en maintenant une approximation intermédiaire de p_θ, qui décompose le transport en plus petites étapes.

— La contrainte K-Lipschitz garantit que cette approximation reste valide et empêche le paysage énergétique E_θ de faire diverger la dynamique de Langevin.

Ensemble, ils permettent un entraînement stable avec un nombre plus limité de pas MCMC et consolident l'interprétation Wasserstein de la fonction de coût des EBM.

4 Pistes suivies

4.1 Baseline non contrainte

Notre baseline est un EBM standard entraîné par dynamique de Langevin de 100 pas par itération, avec buffer [10]. L'algorithme d'entraînement est décrit ci-dessous.

Algorithm 2 Entraînement d'EBM (MCMC à courte durée avec buffer)

Require: Jeu de données $\mathcal{D}$, réseau d'énergie E_θ, ϵ, nombre de pas MCMC λ, batch size N

Instancier un buffer $\mathcal{B} \leftarrow \mathcal{N}(\mathbf{0}, \mathbf{I})$
for chaque itération d'entraînement **do**
  Échantillonnage d'images réelles : $\{\mathbf{x}^+\} \sim p_{\mathcal{D}}$
  Initialiser les échantillons négatifs : $\mathbf{x}_0^- \sim \mathcal{B}$ pour 95% des cas, $\mathcal{N}(\mathbf{0}, \mathbf{I})$ sinon
  for $t = 1, \ldots, \lambda$ **do**
    Calculer le gradient d'énergie : $\mathbf{g}_t = \nabla_{\mathbf{x}} E_\theta(\mathbf{x}_{t-1}^-)$
    Mettre à jour : $\mathbf{x}_t^- = \mathbf{x}_{t-1}^- - \frac{\epsilon}{2}\,\mathbf{g}_t + \sqrt{\epsilon}\,\boldsymbol{\eta}_t, \quad \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  end for
  — **Mise à jour des paramètres** —
  Calculer la perte contrastive :

$$\mathcal{L} = \mathbb{E}\left[E_\theta(\mathbf{x}^+)\right] - \mathbb{E}\left[E_\theta(\mathbf{x}_\lambda^-)\right]$$

  Mettre à jour $\theta \leftarrow \theta - \alpha\,\nabla_\theta\,\mathcal{L}$
  $\mathcal{B} \leftarrow \mathbf{x}_\lambda^-$ (remplacer les $\mathbf{x}_0^-$ par leurs $\mathbf{x}_\lambda^-$ associés)
end for

Cet apprentissage est donc très long dû aux 100 pas de Langevin nécessaires pour chaque batch à chaque itération.

4.2 Apprentissage Lipschitz

Ainsi, la première piste que nous avons suivie est celle vérifiant nos hypothèses : avec une contrainte de Lipschitz, il est possible d'utiliser beaucoup moins de pas de Langevin et conserver la stabilité. Dans cette partie, nous allons donc fixer nos pas de Langevin à 10. (10 fois moins !)

4.2.1 La bibliothèque `deel-torchlip`

Ici nous ne modifions *ni la fonction de coût, ni la dynamique de Langevin, ni l'algorithme d'entraînement.* La seule modification apportée à la baseline est d'imposer une contrainte de Lipschitz sur le réseau d'énergie E_θ, en remplaçant les couches standards par leurs équivalents certifiés, et en substituant les activations classiques (ReLU, etc.) par `GroupSort2`, qui est 1-Lipschitz. La fonction de coût contrastive reste identique à celle de la baseline :

$$\mathcal{L} = \mathbb{E}_{x^+ \sim p_{\mathcal{D}}}\left[E_\theta(x^+)\right] - \mathbb{E}_{x^- \sim p_\theta}\left[E_\theta(x^-)\right]$$

L'objectif est précisément d'isoler l'impact de la constante de Lipschitz K : en faisant varier K tout en gardant le reste constant, nous pouvons mesurer son effet sur la stabilité de l'entraînement, la qualité des échantillons générés et l'expressivité du modèle.

Pour ça,la bibliothèque `deel-torchlip` (cf section 2.4) est utilisée. Elle fournit des remplaçants directs des couches PyTorch standard qui sont garantis d'être 1-Lipschitz (ou K-Lipschitz avec une constante spécifiée par l'utilisateur). Conçue à l'origine pour la robustesse certifiée en classification, elle est ici réutilisée pour l'entraînement des EBMs.

Principaux composants utilisés : Les couches utilisées dans notre architecture de réseau d'énergie sont :

- `SpectralLinear` : une couche dense dont les poids sont contraints à avoir une norme spectrale ≤ 1 via l'itération de puissance et l'orthogonalisation de Björck.
- `SpectralConv2d` : une couche de convolution Lipschitz par la norme spectrale et l'orthogonalisation du kernel.
- `GroupSort2`.

Fixer $K = 1$ donne un réseau d'énergie 1-Lipschitz ; fixer $K = 10$ relâche la contrainte et permet des paysages énergétiques plus abrupts. Cependant, il est difficile de savoir quelle constante est la plus optimale, elle rajoute donc un nouvel hyper-paramètre à ajuster.

4.3 Heuristique optimale pour déterminer K

Nous cherchons donc à dégager une heuristique pour choisir la constante de Lipschitz K optimale. Considérons la mise à jour de Langevin sur un échantillon x après λ pas de Langevin :

$$x_\lambda = x_0 + \sum_{t=1}^{\lambda} \frac{\epsilon}{2} K \nabla_{\mathbf{x}} E_\theta(\mathbf{x_t}) + \sum_{t=1}^{\lambda} \sqrt{\epsilon}\, \boldsymbol{\eta_t}, \quad \boldsymbol{\eta_t} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

où K est la constante de Lipschitz multiplicative du réseau, et ε le pas de bruit. Or, au fil des itérations, le terme stochastique s'atténue : $\sum_{t=1}^{\lambda} \sqrt{\epsilon}\, \boldsymbol{\eta_t} \to \infty$ (c'est une conséquence de la loi des grands nombres (η est centré), donc la somme converge vers 0 en espérance). Ainsi, la distance parcourue par les λ pas de Langevin, soit la distance entre x_λ et x_0 s'écrit :

$$\|x_\lambda - x_0\| = \left\| \sum_{t=1}^{\lambda} \frac{\epsilon}{2} K \nabla_{\mathbf{x}} E_\theta(\mathbf{x_t}) \right\| \quad \textbf{et} \quad \|x_{t+1} - x_t\| = \frac{\epsilon}{2} K \|\nabla_{\mathbf{x}} E_\theta(\mathbf{x_t})\|$$

Si l'entraînement se passe bien, nous voulons pouvoir réduire parfaitement la distance de Wasserstein $W_1(\mathit{bruit}, \mathit{distribution})$ entre du bruit et notre distribution cible avec des pas de Langevin. Nous proposons donc de choisir K de telle manière que avec λ pas de Langevin de taille $\frac{\epsilon}{2} K \|\nabla_x E_\theta(x)\|$, **cette distance est entièrement parcourue** :

$$W_1(\mathit{bruit}, \mathit{distribution}) \approx \lambda \left(\frac{\epsilon}{2} K \|\nabla_x E_\theta(x)\| \right) \quad \Leftrightarrow \quad K_{\mathit{optimale}} \approx \frac{W_1(\mathit{bruit}, \mathit{distribution})}{\lambda \frac{\epsilon}{2} \|\nabla_x E_\theta(x)\|}$$

Ici, $W_1(\mathit{bruit}, \mathit{distribution})$ est fixée car elle représente une distance entre le bruit et la distribution ciblée, elle doit être **calculée avant l'entraînement**. Il est possible donc en fixant λ, obtenir un K idéal pour parcourir notre paysage énergétique. En effet, si cette contrainte n'est pas respectée, même toujours dirigé dans la bonne direction, le processus ne pourrait pas atteindre la distribution en λ pas à partir de bruit.

Le gradient $\|\nabla_x E_\theta(x)\|$ est mesuré empiriquement sur les échantillons au cours de l'entraînement. L'objectif est donc maintenant de calculer la distance de Wasserstein à parcourir $W_1(\mathit{bruit}, \mathit{distribution})$ et en combien de pas λ nous voulons parcourir cette distance pour pouvoir calculer la constante de Lipschitz K optimale.

→ **Pour $W_1(bruit, distribution)$:** Nous calculons cette distance avant l'entraînement à l'aide de librairies comme POT [13]. Cependant, $W_1(bruit, distribution)$ représente une distance *en ligne droite* entre le bruit et la distribution cible, alors que la chaîne de Langevin ne suit pas une trajectoire rectiligne : à chaque pas, la direction du gradient peut dévier de la direction optimale. Ainsi, même si la norme du pas $K_{optimale} \cdot \|\nabla_x E_\theta(x)\|$ est idéale, la distance effectivement parcourue vers la cible est inférieure à cette norme, et la contrainte n'est valide que dans le cas parfait où le gradient pointe toujours dans la bonne direction.

Remarque 3. *En pratique, le chemin réellement parcouru est plus long que $W_1(bruit, distribution)$, et cette heuristique ne dispose d'aucun mécanisme pour mesurer la distance réellement parcourue, ni pour s'y adapter.*

Cependant d'après l'équation (22), la fonction de coût de notre EBM est (si E_θ est 1-Lipschitz) égale à une approximation de la distance de Wasserstein entre la distribution générée dans le buffer par l'EBM et la distribution cible. C'est-à-dire que cette valeur représente la distance en moyenne entre les échantillons appairés du buffer et du jeu de données cible qu'il nous **reste effectivement à parcourir**. Cette valeur instantanée de la distance de Wasserstein estimée W_1^{estime} nous permet de proposer une adaptation dynamique de la constante de Lipschitz du réseau au cours de l'apprentissage.

Il est ainsi possible de parler d'une *contrainte adaptative* qui adaptera K durant l'entraînement en fonction de la distance parcourue.

$$\boxed{K \approx \frac{W_1^{estime}}{\lambda \frac{\epsilon}{2} \|\nabla_x E_\theta(x)\|}}$$

Cela fournit une heuristique naturelle pour calibrer K par rapport à λ. Un λ trop petit donnerait un K trop grand qui reproduirait les instabilités de la baseline non contrainte. Un λ trop grand, causerait un K trop petit qui ralentirait le transport.

→ **Pour λ :** La référence λ dans les équations précédentes peut être choisie de différentes façons. Il est possible par exemple de prendre le nombre de pas réalisés à chaque itération (10 dans notre apprentissage Lipschitz), ce qui impose donc à notre dynamique de Langevin de pouvoir parcourir la distance de Wasserstein entière à chaque itération. Chose qui n'est pas souhaitable, surtout aux premières itérations où la distribution générée est encore très aléatoire et bruitée, et qui pourrait donner ainsi des valeurs de notre constante de Lipschitz trop grandes.

Il existe cependant une autre piste qui prend en compte la mécanique du buffer et de ses réinitialisations aléatoires. En effet, le buffer est raffiné à chaque itération de λ pas, et un certain pourcentage des échantillons est réinitialisé à partir du bruit. Cette réinitialisation impose donc une limite du nombre de pas de Langevin moyen vus par les éléments du buffer.

Idée clé

Chaque échantillon voit 10 pas de Langevin par itération et a 5% de chance d'être remplacé par du bruit. Nous pouvons donc calculer le nombre moyen de pas de Langevin que le buffer va atteindre un maximum de pas vus à cause de la réinsertion de bruit dans ce dernier.

La section suivante donne une estimation du nombre moyen de pas de Langevin effectués sur les éléments du buffer.

4.3.1 Évaluation du nombre de pas de Langevin et critère d'arrêt d'entraînement

Nous cherchons donc à étudier le nombre de pas de Langevin que chaque élément du buffer va effectuer, et en déterminer la moyenne en calculant la limite lorsque le nombre de pas d'apprentissage tend vers l'infini. Pour cela, nous modélisons la dynamique de la moyenne de pas de Langevin vus par le buffer comme m_n au fil des itérations n.

Posons b correspondant à la **batch size** sur laquelle nous travaillons (le nombre d'éléments traités et mis à jour par itération), N la **taille du replay buffer**, r un **taux de réinitialisation** par du bruit et λ le **nombre de pas de Langevin** par itération. À chaque itération, b éléments sont sélectionnés : les br éléments réinitialisés voient leur contribution moyenne m_n supprimée, tandis que les b éléments sélectionnés effectuent chacun λ nouveaux pas de Langevin. Nous posons la récurrence :

$$m_{n+1} = \left(1 - \frac{br}{N}\right) m_n + \frac{b}{N}\lambda$$

En posant

$$a = 1 - \frac{br}{N}, \qquad c = \frac{b}{N}\lambda,$$

Nous obtenons la suite affine récurrente d'ordre 1 : $m_{n+1} = a\, m_n + c$.

En itérant depuis $m_0 = 0$:

$$m_n = a^n m_0 + c \sum_{k=0}^{n-1} a^k = c\, \frac{1 - a^n}{1 - a}.$$

Et en substituant a et c :

$$\boxed{m_n = \frac{\lambda}{r}\left(1 - \left(1 - \frac{br}{N}\right)^n\right)} \tag{23}$$

Comme $\left|1 - \frac{br}{N}\right| < 1$, la suite converge et :

$$\lim_{n\to\infty} m_n = \frac{\lambda}{r}.$$

La moyenne du nombre de pas de Langevin vus dans le buffer donc vers un palier déterminé uniquement par les pas de Langevin λ et le taux r.

Exemple

Prenons un exemple type :

1. $b = 100$
2. $N = 50000$ (CIFAR10)
3. $r = 0.05$ soit 5% de réinitialisation
4. $\lambda = 10$ pas de Langevin par itération

Nous obtenons donc à l'itération $n = 40000$, $m_n \approx 196$ pas de Langevin moyen
Et pour l'itération $n = \text{inf}$, $m_n = 200$ pas de Langevin moyen

En réalisant la soustraction entre le nombre de pas moyen vu par le buffer à l'itération t et la limite $m = \lambda/r$, nous pouvons calculer le nombre moyen de pas de Langevin que les éléments du buffer peuvent encore faire.
Il nous revient de trouver l'itération n à laquelle nous voulons stopper l'entraînement.

4.3.2 Critère d'arrêt d'entraînement

Comme vu plus tôt un EBM apprend avec une fonction de coût qui oscille (Figure 6), il n'est donc pas évident de savoir quand est-ce que la convergence du modèle est atteinte.

Pour un EBM 1-Lipschitz, la fonction de coût représente une distance de Wasserstein (comme vu plus haut) entre la distribution générée et la distribution cible.

La fonction de coût devrait donc en théorie converger vers 0, le modèle étant meilleur au fil des itérations, améliorant ainsi sa dynamique de Langevin implicitement. Ainsi la fonction de coût pourrait nous indiquer un critère d'arrêt facile et interprétable.

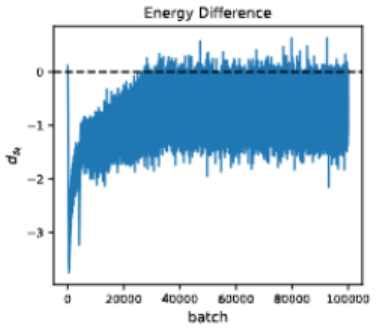


FIGURE 11 – Loss d'un EBM 1-Lip par itération

Sur la Figure 11 nous pouvons observer que la fonction de coût atteint un plafond vers les 40 000 itérations, ce qui implique que l'EBM n'arrive plus à raffiner encore son buffer et à le rapprocher de la distribution cible.

Il est possible d'expliquer ce comportement avec notre exemple pris plus tôt, avec les mêmes paramètres que ceux utilisés pour la Figure 11, nous obtenons avec l'équation (23) que :

— Pour $n = 40000$, $m_n \approx 196.3$ pas de Langevin moyen

— Pour $n = 100000$, $m_n \approx 199.9$ pas de Langevin moyen

Ainsi, le nombre de pas moyen entre les deux itération est très proche ! Il nous faudrait faire 60 000 itérations en plus, pour pouvoir rajouter en moyenne 3 pas de Langevin seulement, ce qui semble déraisonnable et qui peut donc nous indiquer dans ce cas que 40 000 itérations est un palier clé pour l'entraînement.

Ce palier constitue un critère de convergence de l'entraînement, et permet de fixer une itération n finale, et qui nous permet de calculer notre constante K idéale pour chaque itération t avec $t < n$.

Exemple

Nous souhaitons faire un entraînement de 40 000 itérations. Lors de l'itération n = 6 000, avec les mêmes paramètres des exemples précédents, $\epsilon = 2$ et la constante de Lipschitz courante $K_{6000} = 1$

— Les éléments du buffer ont vus en moyenne $m_{6000} = 90.2$ pas de Langevin (cf Eq.23). Ces mêmes éléments pourront voir un maximum moyen de $196.3 - 90.2 = 106.1$ pas de Langevin supplémentaires.

— La fonction de coût à l'itération 6000 est égale à 120 donc $W_1^{\text{estimée}} = \frac{120}{K_{6000}} = 120$

— Le norme moyenne du gradient sur les pas de Langevin est estimée à 0.7 (ce qui vérifie bien la propriété 1-Lipschitz) ($\|\nabla_x E_\theta(x)\|$)

Il devient donc possible de calculer une constante de Lipschitz idéale : $K = \frac{120}{106.1*1*0.7} = 1.61$. Avec cette contrainte de Lipschitz, la dynamique de Langevin peut en théorie parcourir toute la distance nécessaire.

Remarque 4. *Ces résultats sont très approximatifs et sous-évaluent beaucoup de paramètres du problème :*

1. *Le modèle 1-Lip n'est pas exactement 1 Lipschitz en raison de la difficulté de la tâche, le modèle est plus probablement $\leq$ 1-Lip*
2. *$W_1^{estimée}$ est une valeur approchée de la distance de Wasserstein, calculée par la fonction de coût d'un modèle supposé 1-Lip. Comme vu plus tôt, si le modèle est sous évalué et est K-Lip avec $K < 1$, alors :*

$$W_1^{estimée} = K * D_W^{bruit \to distribution} < D_W^{bruit \to distribution}$$

   Ce qui revient donc à calculer très probablement une approximation sous-évaluée de la vraie distance de Wasserstein

3. *Le nombre de pas de Langevin moyen m_n vus par le buffer est une moyenne et n'est pas exact.*

Nous avons donc vérifié les hypothèses empiriquement et prévu un facteur à la constante pour compenser les sous-évaluations (ex. $K_{new} = K * 2$)

Nous obtenons finalement la formule suivante :

$$\boxed{K_{t+1} \approx 2 \cdot \frac{W_1^{\text{estimée}}}{(m_n - m_t)\frac{\epsilon}{2} \|\nabla_x E_{\theta_t}(x)\|}} \quad \text{avec } t < n \tag{24}$$

Avec ceci, nous essayons d'avoir une constante de Lipschitz capable de nous emmener jusqu'à notre distribution en fonction du rythme actuel de notre Langevin.

4.4 Architecture à double réseau avec pré-initialisation Lipschitz

Nous explorons enfin l'hypothèse qu'une contrainte de Lipschitz stricte n'est pas nécessaire sur l'intégralité de l'entraînement. L'idée centrale est de découpler l'entraînement en deux phases successives, en entraînant de façon séquentielle deux réseaux distincts.

Comme évoqué précédemment, un EBM est d'autant plus puissant qu'il peut se déplacer efficacement vers sa distribution cible. En partant de bruit, un modèle non contraint a tendance à diverger : sans un grand nombre de pas de Langevin à chaque itération, il est incapable de converger correctement vers sa cible.

À l'inverse, un modèle contraint (Lipschitz) reste stable et peut accomplir sa tâche même avec très peu de pas de Langevin. Sa stabilité induit cependant une expressivité limitée qui l'empêche de capturer parfaitement la distribution cible, ce qui bride la qualité des images générées. Une solution naturelle est alors de combiner les deux approches en deux phases :

— **Un premier EBM Lipschitz** est entraîné sur de bruit uniforme avec peu de pas de Langevin. . Grâce à sa stabilité, il explore sans difficulté les régions les plus complexes de l'espace et produit un résultat plus facilement interprétable.

— **Un deuxième EBM non contraint classique** est entraîné, mais en guidant la phase de réinitialisation des éléments du buffer par le réseau 1-Lipschitz.

Idée clé

Cette approche permet de n'avoir dans le buffer du deuxieme EBM, que des images plus proches de la distribution que le bruit uniforme et permet au deuxième EBM d'affiner le résultat jusqu'à la distribution cible. (cf Figure 12).

Les pas de la chaîne de Markov du modèle Lipschitz sont plus erratiques car elle explore une distribution encore très éloignée de la cible. Cependant, grâce à la contrainte de Lipschitz, ces pas ne feront jamais exploser l'apprentissage : leurs normes sont bornées (≤ 1), ce qui les rend petits et contrôlés. À l'inverse, le modèle non contraint E_ϕ produit des pas plus grands et plus forts, mais ceux-ci restent stables puisque le buffer a déjà été initialisé par plusieurs pas de Langevin suivant le gradient de E_θ^{Lip} (cf Figure 12).

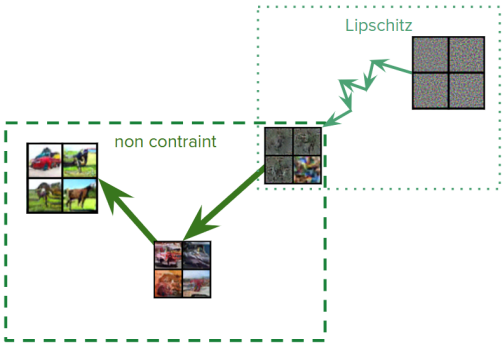


FIGURE 12 – Exemple visuel de pré-initialisation Lipschitz

4.4.1 Phase de pré-initialisation : modèle Lipschitz.

Dans un premier temps, seul un modèle d'énergie contraint E_θ^{Lip} est entraîné, pendant un nombre réduit d'itérations. Cette *pré-initialisation* a pour unique vocation de peupler le buffer de replay avec des échantillons de qualité suffisante : grâce aux résultats établis à la section précédente, il est possible de savoir que chaque élément du buffer a subi en moyenne m_n pas de Langevin depuis le bruit initial. Ces m_n pas constituent une chaîne déjà bien avancée vers la distribution cible, et représentent un point de départ bien plus informatif que le bruit pur.

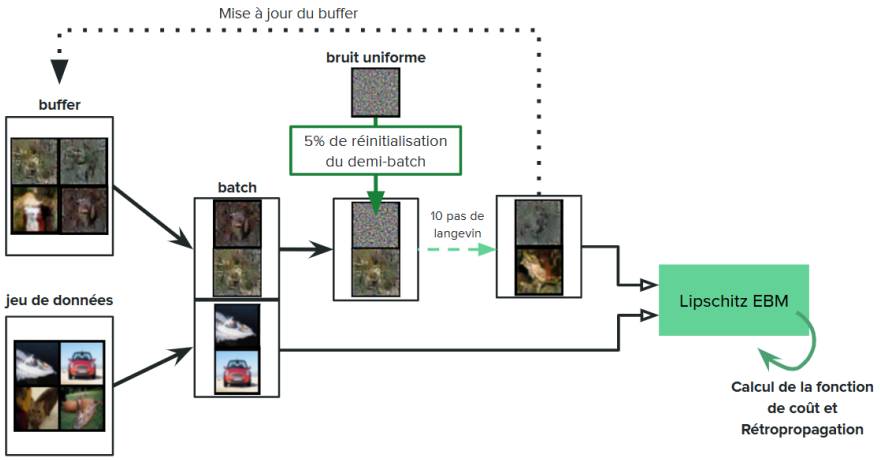


FIGURE 13 – Exemple d'un cycle d'entraînement d'un modèle Lipschitz E_θ^{Lip}

4.4.2 Phase principale : modèle non contraint.

Dans un second temps, un second modèle E_ϕ est entraîné *sans* contrainte de Lipschitz. Cependant, contrairement à la méthode classique où une fraction du buffer est périodiquement réinitialisée avec du bruit pur (5% dans nos exemples, cf algorithme 2). Ici E_ϕ **ne démarre jamais depuis du bruit**, mais toujours depuis des échantillons déjà produits par le modèle contraint E_θ^{Lip}.

Concrètement, le buffer de E_ϕ est entièrement alimenté par E_θ^{Lip} : à l'initialisation de E_ϕ, ce dernier récupère le buffer raffiné pendant l'entraînement de E_θ^{Lip}. Puis, lorsqu'un élément doit être réinitialisé, plutôt que de le remplacer par du bruit, est appliqué m_n pas de Langevin guidés par E_θ^{Lip} à partir de bruit (avec m_n le nombre de pas que son buffer a vu, pour générer des échantillons similaires au buffer).

Puisque les éléments du buffer sont déjà proches de la distribution cible après la pré-initialisation Lipschitz, le modèle non contraint E_ϕ peut ainsi être entraîné de façon stable même avec un faible nombre de pas de Langevin (contrairement à avant !).

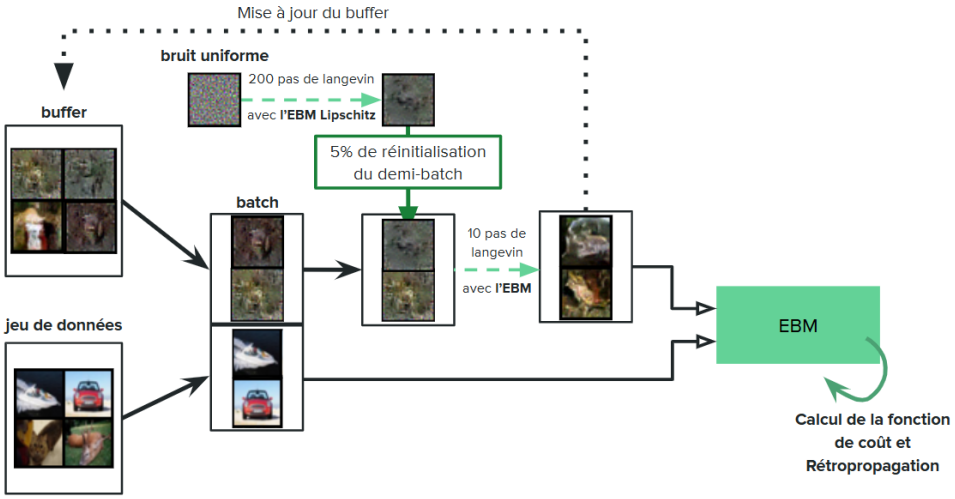


FIGURE 14 – Exemple d'un cycle d'entraînement du pipeline pré-initialisation Lipschitz

Remarque 5. *Cette procédure implique que le modèle final $E_{\phi,\theta}$ n'est* pas *un véritable EBM au sens strict : il ne définit pas à lui seul une distribution de Boltzmann–Gibbs normalisable de la forme présentée en (1), puisque la génération de ses exemples négatifs repose sur un second réseau E_θ^{Lip} externe. En d'autres termes, la théorie derrière la distribution représentée par notre EBM est moins claire et fondée. De plus, il n'est plus possible d'obtenir facilement l'énergie d'un échantillon. (voir nos pistes de solutions à ce problème en Annexe C.1)*

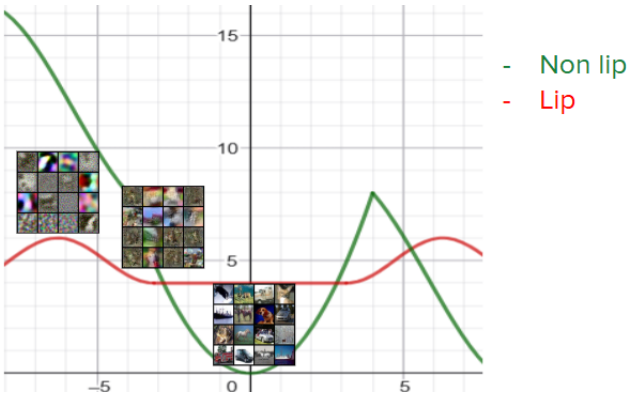


FIGURE 15 – Exemple d'un paysage énergétique d'un pipeline de pré-initialisation Lipschitz

Comme visible sur la figure 15, le paysage énergétique du modèle non contraint E_ϕ (en vert) est bien plus expressif que celui du modèle E_θ^{Lip} (en rouge) : les minima sont plus profonds et mieux définis, ce qui correspond à de meilleurs échantillons générés (cf. images sur la Figure 15).

En revanche, si nous entraînions E_ϕ directement à partir de bruit, la chaîne de Markov se perdrait dans ce paysage complexe et divergerait. C'est justement pour ça qu'est utilisée la pré-initialisation : E_θ^{Lip} fournit un buffer déjà propre, ce qui évite à E_ϕ de voir du vrai bruit pendant l'entraînement.

Ainsi, avec la pré-initialisation, si du bruit est directement soumis à E_ϕ au moment de l'inférence, ce dernier renverra donc une énergie complètement aléatoire et inexploitable, puisqu'il n'a tout simplement jamais été entraîné dans ces régions de l'espace. Le schéma génératif consiste donc à appliquer sur le bruit m_n pas de Langevin guidés par E^{Lip}, puis les autres pas de Langevin guidés par E_ϕ.

Cette architecture à deux réseaux séquentiels reste cependant coûteuse et peu élégante ; nous avons donc cherché à la rendre plus propre en fusionnant les deux modèles en une architecture unifiée, présentée en Annexe C.1. Une autre solution est présentée section suivante, plus simple et plus pratique.

4.5 Semi-Lipschitz

Les deux approches précédentes imposent la contrainte de Lipschitz de façon globale : soit sur l'intégralité du réseau d'énergie (section 4.2), soit sur un second réseau entraîné séparément en amont (section 4.4). Nous proposons ici une troisième piste, plus légère à mettre en œuvre : contraindre uniquement une **partie** du réseau d'énergie E_θ, tout en conservant un unique réseau entraîné de bout en bout.

Idée clé

Contrairement aux méthodes précédentes, l'algorithme d'entraînement utilisé pour cette méthode est *exactement* l'algorithme 2 de la baseline. Le Semi-Lipschitz est une modification purement **structurelle** du réseau E_θ, et non une nouvelle procédure d'apprentissage.

4.5.1 Architecture

Décomposons le réseau d'énergie de L couches en 3 blocs :

$$E_\theta = h_\phi \circ f_\gamma \circ f_\psi, \qquad \theta = (\phi, \gamma, \psi) \tag{25}$$

où :

- $f_\psi : \mathbb{R}^n \to \mathbb{R}^m$ est un **extracteur de caractéristiques**, composé des L_1 premières couches du réseau. Ces couches sont remplacées par leurs équivalents `deel-torchlip` (`SpectralConv2d`, `SpectralLinear`) et utilisent l'activation `GroupSort2`, ce qui rend f_ψ strictement 1-Lipschitz ;
- $f_\gamma : \mathbb{R}^m \to \mathbb{R}^d$ est aussi un **extracteur de caractéristiques**, composé des L_2 couches du réseau. Ces couches sont non contraintes et permettent l'extraction des caractéristiques de l'espace latent construit par f_ψ
- $h_\phi : \mathbb{R}^d \to \mathbb{R}$ est une **tête**, composée des L_3 dernières couches, qui reste **non contrainte**.

Avec : $L = L_1 + L_2 + L_3$ et $n > m > d$

Autrement dit, seul le premier bloc f_ψ, celui qui traite en premier le signal x, est rendu Lipschitz ; les blocs suivants, f_γ et h_ϕ, qui opèrent respectivement sur l'espace intermédiaire $f_\psi(x)$ et sur l'espace des caractéristiques $z = f_\gamma(f_\psi(x))$, restent libres.

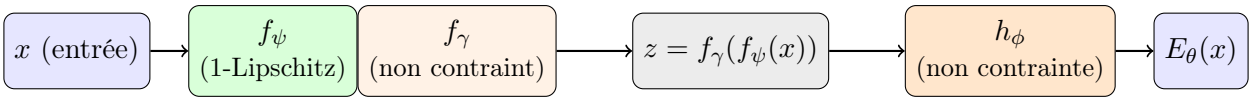


FIGURE 16 – Architecture Semi-Lipschitz : un extracteur de caractéristiques f_ψ contraint à être 1-Lipschitz, suivi d'un autre extracteur f_γ et d'une tête h_ϕ non contraints qui produisent l'énergie finale $E_\theta(x)$.

4.5.2 Découpage architectural

Ce choix architectural découle directement des hypothèses posées aux sections 3.1.1, 3.1.2 et 3.1.3, en identifiant où, dans le réseau, la contrainte de Lipschitz apporte le plus de stabilité pour le moins de coût en expressivité.

Stabilité. La contrainte 1-Lipschitz sur f_ψ implique que $\|f_\psi(x_1) - f_\psi(x_2)\| \leq \|x_1 - x_2\|$. Autrement dit, f_ψ ne peut pas éloigner deux points au-delà de leur distance dans l'espace d'entrée : deux points proches en entrée restent, après passage par f_ψ, à une distance au plus égale à leur distance d'origine. L'espace latent construit par f_ψ est donc à la fois de dimension plus faible que l'espace d'entrée, et muni d'une structure de distances directement contrôlée par la contrainte 1-Lipschitz : la variation de $f_\psi(x)$ induite par toute perturbation de x est bornée par la norme de cette perturbation. Le reste du réseau, f_γ et h_ϕ, opère ainsi sur un espace de dimension réduite dont les variations sont contrôlées.

Expressivité. Le reste du réseau, composé de f_γ et h_ϕ, n'est soumis à aucune contrainte de Lipschitz. La contrainte 1-Lipschitz porte uniquement sur f_ψ : elle ne borne ni la norme des poids de f_γ et h_ϕ, ni leur constante de Lipschitz respective. Par construction, f_γ et h_ϕ n'étant soumis à aucune contrainte de Lipschitz, rien n'empêche ces deux blocs de réaliser des fonctions bien plus expressives et *sharp* sur l'espace latent $z = f_\gamma(f_\psi(x))$. La capacité de E_θ à produire des variations locales importantes de l'énergie n'est donc pas limitée par la contrainte structurelle imposée à f_ψ, à la différence des architectures des sections 4.2 et 4.4, où l'ensemble du réseau d'énergie (respectivement un second réseau entraîné séparément) est contraint.

Remarque 6. *Cette architecture ne garantit* pas *que E_θ dans son ensemble soit K-Lipschitz : la borne* $\mathrm{Lip}(E_\theta) \leq \prod_l \mathrm{Lip}(f_l)$ *reste non contrôlée dès lors que f_γ et h_ϕ sont libres. L'interprétation de la fonction de coût d'un EBM avec la distance de Wasserstein (section 3.1.2 et Remarque 2) est ainsi perdue.*

4.5.3 Choix de la profondeur contrainte L_1

Le nombre de couches contraintes L_1 devient un nouvel hyper-paramètre du corps f_ψ :

- $L_1 \to 0$: aucune couche contrainte, nous retrouvons le réseau non contraint de la baseline (section 5).
- $0 < L_1$, $L_2 = 0$: l'extracteur de caractéristiques est entièrement 1-Lipschitz.
- $0 < L_1$, $L_2 > 0$: l'extracteur de caractéristiques est partagé entre un corps 1-Lipschitz (L_1 couches) et un corps non contraint (L_2 couches), c'est le cas Semi-Lipschitz étudié dans cette section.

Plus L_1 est grand, plus nous nous rapprochons du comportement stable mais moins expressif d'un réseau totalement Lipschitz ; plus L_1 est petit, plus nous nous rapprochons de l'expressivité de la baseline mais aussi de son instabilité.

5 Protocole expérimental

5.1 Jeu de données

Toutes les expériences sont menées sur **MNIST** : 60 000 images de chiffres manuscrits en niveaux de gris de 28×28 pixels. MNIST est un référentiel bien compris qui permet une itération rapide et des vérifications visuelles de cohérence. Cependant ce dataset est très permissif et peut faire fonctionner des méthodes en réalité non-rapportable à des jeux de données beaucoup plus complexes.

Nous utilisons donc également le dataset **CIFAR-10**, composé de 60 000 images en couleur de taille 32×32 pixels réparties en 10 classes (avions, automobiles, oiseaux, chats, cerfs, chiens, grenouilles, chevaux, navires et camions). Contrairement à MNIST, les images présentent une forte variabilité visuelle : arrière-plans complexes, objets observés sous différents angles, variations d'éclairage et de texture, RGB et non pas en niveaux de gris...

5.2 Architecture

Pour nos expériences, nous utilisons un **WideResNet-16** [52] standard et nous comparons plusieurs méthodes d'entraînement :

— **Baseline non contrainte** : entraînement avec 100 pas de Langevin par itération (cf. algorithme 2).

— **Réseau Lipschitz** : entraînement sous contrainte de Lipschitz avec 10 pas de Langevin par itération, pour différentes valeurs de constante de Lipschitz K (même algorithme qu'au-dessus, mais en imposant une contrainte K au réseau).

— **Pré-initialisation** : deux modèles sont entraînés séparément, un modèle 1-Lipschitz et un modèle non contraint, chacun avec 10 pas de Langevin. Lors de l'entraînement du modèle non contraint, le buffer est réinitialisé à partir de 200 pas de Langevin générés par le modèle Lipschitz, ce qui correspond au nombre de pas observé dans le buffer du modèle Lipschitz à la fin de son entraînement (cf. Figure 14).

— **Semi-Lipschitz** : un unique modèle est entraîné de bout en bout avec le même algorithme que la baseline (algorithme 2), avec 10 pas de Langevin par itération. Seules les L_1 premières couches du réseau d'énergie sont contraintes à être 1-Lipschitz.

5.2.1 Initialisation et réglages des hyper-paramètres

Hyper-paramètre	Symbole	Valeur
Pas MCMC	λ	10 ou 100
Pas de Langevin	ϵ	2
Optimiseur	–	Adam, $\alpha = 10^{-4}$
Batchsize	b	100
Constante de Lipschitz	K	1–100
Itérations de Björck	n_{iter}	10

TABLE 1 – hyper-paramètres variés dans l'étude expérimentale.

5.3 Métriques

L'évaluation des images générées est un problème délicat. Nous utilisons principalement le **Fréchet Inception Distance (FID)**, calculé à partir d'un encodeur **InceptionV3**.

Le FID mesure la distance entre deux distributions gaussiennes ajustées aux représentations des images réelles et générées dans un espace de caractéristiques. En effet, nous supposons que les activations extraites (par InceptionV3 ou un encodeur équivalent) suivent des lois gaussiennes :

$$\mathcal{N}(\mu_r, \Sigma_r) \quad \text{et} \quad \mathcal{N}(\mu_g, \Sigma_g),$$

où μ_r, Σ_r sont la moyenne et la covariance des caractéristiques des images réelles, et μ_g, Σ_g celles des images générées.

Le FID s'écrit alors :

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right).$$

Intuitivement, le premier terme compare les centres des deux distributions (qualité moyenne des caractéristiques), tandis que le second compare leur dispersion et leurs corrélations (diversité et structure des échantillons). Un FID faible indique que les distributions des images générées et réelles sont proches dans l'espace de représentation. Le FID est très critiqué dans la communauté scientifique [21] donc il est important d'approfondir en détail la qualité de nos résultats afin de s'assurer que tout se passe bien.

Pour CIFAR-10, ces représentations sont directement extraites via un réseau InceptionV3 pré-entraîné sur ImageNet. Pour MNIST, nous utilisons une approche adaptée inspirée des travaux de Hrusanov et al. [20] : un réseau de neurones convolutionnel est entraîné sur la tâche de classification, puis sa tête de classification est supprimée afin d'extraire des représentations latentes, sur lesquelles la distance de Fréchet est ensuite calculée.

Afin de suivre le FID de manière cohérente, nous calculerons un FID enregistré à chaque itération, obtenu via une *long-run* de 1000 pas de débruitage à partir d'un bruit pur. Cet indicateur permettra de vérifier que les échantillons sont qualitatifs et convergent bien vers un minimum du paysage énergétique de l'EBM (même avec plus de pas que ce que le buffer à vu) et qu'il y reste de manière stable.

En complément, des inspections visuelles des échantillons générés sont réalisées à intervalles réguliers afin d'évaluer qualitativement la cohérence et la diversité des images produites.

Nous utilisons également plusieurs métriques de diagnostic pour le suivi de l'entraînement :

— l'**écart énergétique** ΔE,

— la **norme du gradient** $\|\nabla_{\mathbf{x}} E(\mathbf{x}^-)\|$ lors des étapes de MCMC,

La notion d'instabilité dans ce contexte reste en partie empirique. Elle est généralement associée à une divergence numérique, caractérisée par des explosions des normes de gradients ou une dérive de l'écart énergétique. Nous considérons donc un entraînement instable dès lors que ces quantités divergent ou font faire une explosion des gradients au cours de l'optimisation.

Ces différents diagnostics permettent d'identifier les instabilités d'entraînement et de distinguer les limitations liées à l'optimisation de celles liées à la capacité du modèle.

6 Résultats et analyses

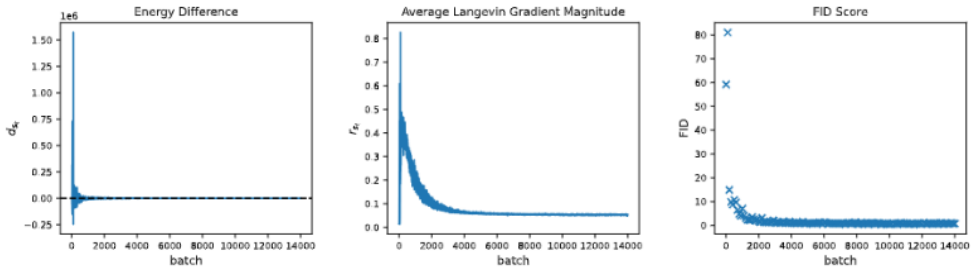
Dans cette section nous analyserons les résultats obtenus et nous vérifierons les hypothèses posées à l'aide des métriques suivies (FID et les courbes d'apprentissage).

6.1 Base line non contrainte

Notre baseline est donc un WideResNet-16 non contraint avec 100 pas de Langevin par itération.

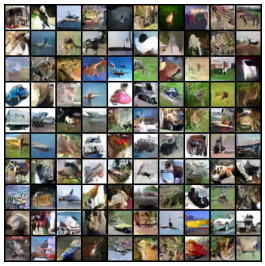


(a) Buffer

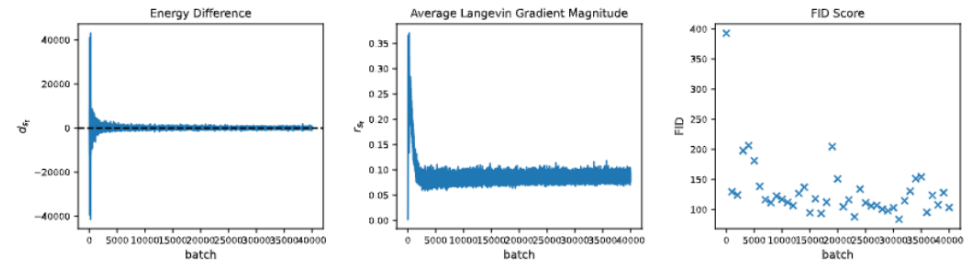


(b) Loss de l'EBM, gradient sur la descente de Langevin, FID d'une *long-run* = 4.8 (1000 steps à partir de bruit pur)

FIGURE 17 – Résultats de la baseline sur MNIST



(a) Buffer



(b) Loss de l'EBM, gradient sur la descente de Langevin, FID d'une *long-run* = 46 (1000 steps à partir de bruit pur)

FIGURE 18 – Résultats de la baseline sur CIFAR10

Comme expliqué plus tôt, la fonction de coût de l'EBM a bel et bien une forme oscillante autour de 0, avec un gradient du Langevin qui se stabilise très vite mais qui continue d'améliorer les générations sans indicateur clair de convergence. Nous pouvons aussi remarquer que au bout de 40 000 itérations (c'est-à-dire quand le buffer a presque vu le maximum de pas de Langevin possible sur lui même, cf Eq. 23), le FID cesse de vraiment s'améliorer ce qui motive l'intérêt de notre critère d'arrêt (ce critère est nuancé et discuté en Annexe E).

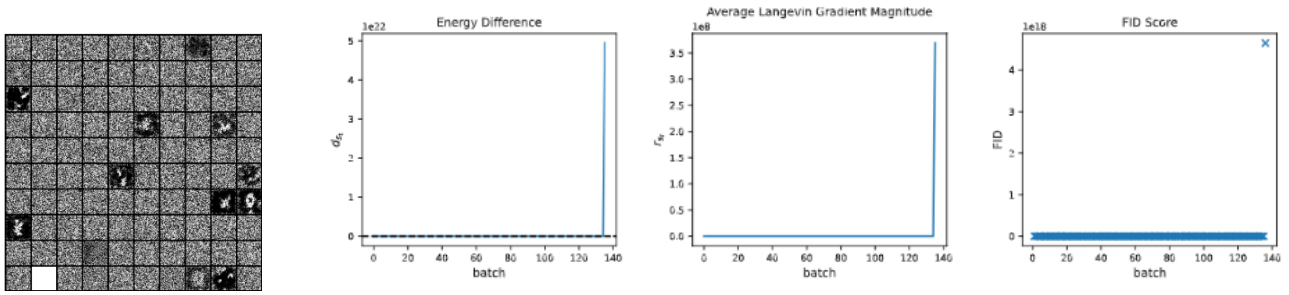
Les classes des images générées sont distinguables pour la plupart, ce qui confirme que l'EBM comprend bien la distribution cible ! Cependant, l'entraînement fut très long et instable, les hyper-paramètres sont très importants et peuvent très vite faire exploser l'entraînement. Nous pouvons aussi remarquer que la tâche d'optimisation est bien plus facile sur MNIST que sur CIFAR10 en raison de la complexité du dataset : couleurs, classes, etc... C'est probablement pour ça qu'avant même que l'EBM n'atteigne son seuil de pas maximum dans son buffer, ce dernier stagne déjà car le problème est trop facile à résoudre.

Remarque 7. *Le FID visible dans la Figure 18b est souvent différent de celui du buffer, ce qui prouve que le modèle a des difficultés à rester stable sur un dataset complexe. (voir Annexe A)*

6.2 Réseau Lipschitz

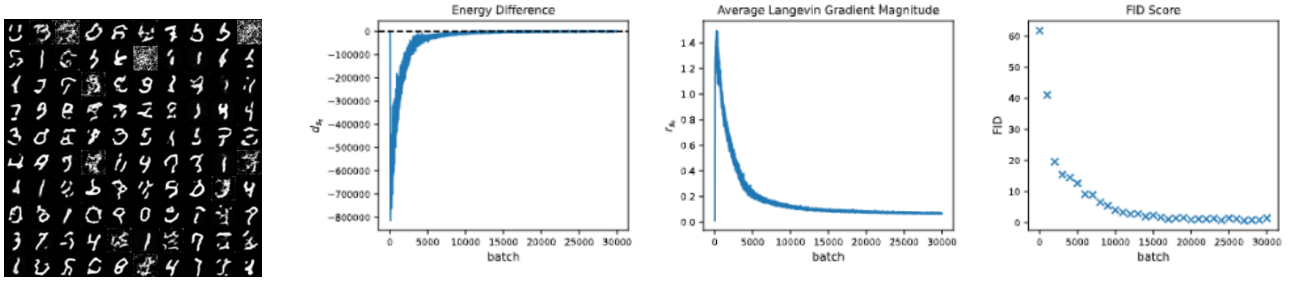
6.3 1-Lipschitz

Tout d'abord, nous voulons vérifier qu'il est possible de remplacer les 100 pas de Langevin de la baseline par seulement 10 pas, en compensant l'instabilité par une contrainte 1-Lipschitz :



(a) Buffer (b) Loss de l'EBM, gradient sur la descente de Langevin, FID d'une *long-run*

FIGURE 19 – Résultats de la baseline avec uniquement 10 pas de Langevin par itération



(a) Buffer 1-lip (b) Loss de l'EBM, gradient sur la descente de Langevin, FID d'une longuerun

FIGURE 20 – Résultats de la méthode 1-lip avec 10 pas de Langevin par itération

Sur MNIST, nos hypothèses semblent valides : un modèle non contraint entraîné avec 10 pas de Langevin diverge immédiatement ; les gradients explosent dès les premières itérations, confirmant que le nombre de pas est le facteur limitant de la baseline.

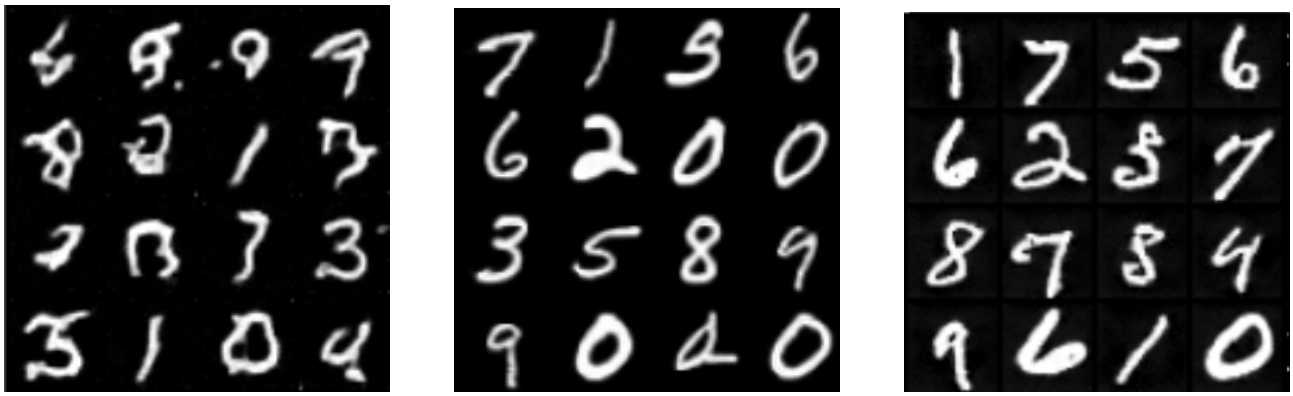
En revanche, le modèle 1-Lipschitz avec 10 pas par itération reste parfaitement stable, l'entraînement ne diverge pas, les hyper-paramètres ont peu d'influence, et les courbes d'apprentissage sont plus nettes. L'entraînement est de surcroît dix fois plus rapide (en termes d'opérations, pas du nombre d'itérations pour atteindre la convergence).

Il est à noter, cependant, que la qualité des images générées est limitée. La contrainte 1-Lipschitz impose un paysage énergétique très lisse, ce qui facilite les pas de Langevin, mais réduit la capacité du modèle à capturer les modes fins de la distribution. Ce premier résultat valide les hypothèses de la section 3.1.1, la contrainte Lipschitz apporte bien la stabilité attendue et ouvre la question du calibrage optimal de K.

Avant donc de passer à un problème plus complexe il nous faut comprendre plus profondément l'impact de la constante K sur notre système.

6.3.1 K-Lipschitz

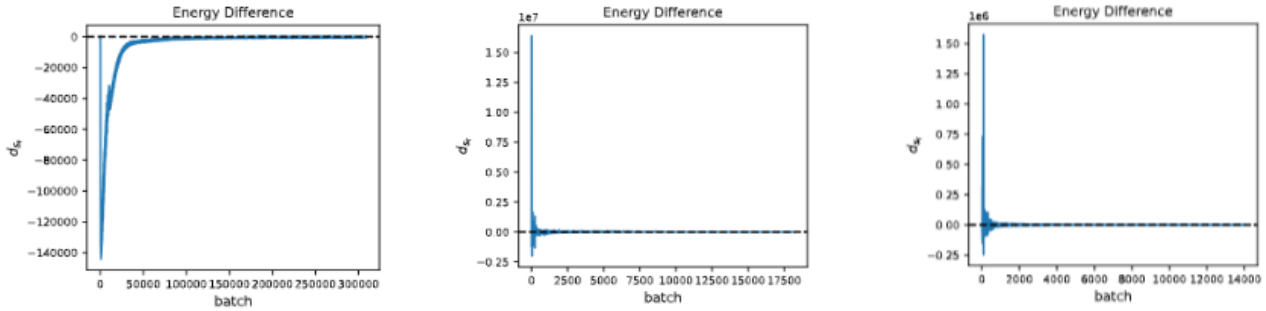
Nous étudions donc maintenant l'effet de la constante K sur MNIST, en comparant $K \in 1, 10, 100$:



(a) 1-lip FID = 11.3 (b) 10-lip FID = 4.2 (c) 100-lip FID = 3.9

FIGURE 21 – Comparaison de qualité entre 1-lip, 10-lip et 100-lip sur une *long-run*

Les résultats sont cohérents avec le compromis expressivité/stabilité décrit en section 3.1.3. $K = 1$ donne les images les moins précises, tandis que K = 100 donne des images plus détaillées. Nous notons qu'un meilleur FID est obtenu par rapport à la baseline avec une contrainte de Lipschitz égale à 10. K semble être donc un très bon paramètre pour ajuster l'expressivité d'un réseau Lipschitz. Cependant, il nous faut aussi étudier la stabilité de ces réseaux :



(a) 10-lip (b) 100-lip (c) Non contraint

FIGURE 22 – Comparaison de la stabilité de la fonction de coût entre un réseau 10-lip, 100-lip avec 10 pas de Langevin et non contraint avec 100 pas de Langevin

Nous observons que K=100 induit un comportement presque identique à celui d'un réseau non contraint. En effet, une contrainte de Lipschitz = ∞ ramène à un système non contraint et donc avec K=100, le réseau est beaucoup plus instable et dépendant des hyper-paramètres. Nous remarquons aussi que plus K est bas, plus la fonction de coût est contrôlée et plus elle représente fidèlement la distance de Wasserstein.

Dans ce cas, nous pouvons établir par exemple que K = 10 représente un compromis intermédiaire acceptable sur MNIST.

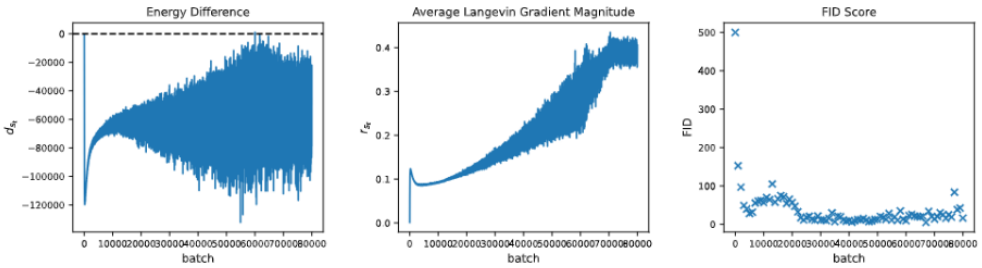
Ce résultat montre que K est un hyper-paramètre à part entière, dont le réglage conditionne directement la qualité du modèle, et que le choisir manuellement n'est pas satisfaisant. C'est ce qui motive l'heuristique de calibrage adaptatif développée en section 4.3.

6.3.2 $K_{Optimal}$-Lipschitz

Nous présentons maintenant les résultats obtenus avec l'heuristique adaptative (Eq. 24) qui ajuste K à chaque itération en fonction de la distance de Wasserstein estimée restant à parcourir et du nombre de pas de Langevin encore disponibles dans le buffer :



(a) Buffer Lipschitz



(b) Loss de l'EBM, gradient sur la descente de Langevin, FID d'une *long-run* = 3.98 (1000 steps à partir de bruit pur)

FIGURE 23 – Résultats de la méthode Lipschitz avec un K optimal sur MNIST

Le FID obtenu est très bon et dépasse encore une fois celui de la baseline, mais cette méthode nous donne l'avantage de ne pas avoir à gérer le paramètre de la constante de Lipschitz. Sur la Figure 23b, nous notons que la magnitude moyenne du gradient du Langevin est de moins en moins contrainte, comme prévu, le modèle est de moins en moins contraint afin de pouvoir correctement arriver à la distribution cible au fil des itérations. La fonction de coût et l'évolution du FID sont stables, ce qui encourage fortement l'utilisation de cette méthode.

Comme visible sur la Figure 24 ci-contre, K se calibre automatiquement en fonction du besoin du Langevin.

Au tout début, il reste beaucoup de pas de Langevin disponibles dans le buffer, donc la contrainte impose une forte contrainte de Lipschitz pour conserver de la stabilité. Au fur et à mesure que les itérations défilent, l'heuristique pousse le modèle et augmente sa contrainte pour le pousser à se déplacer plus vite.

Quand l'heuristique a permis de traverser la distance de Wasserstein, elle stabilise la contrainte et continue l'exploration car le modèle n'atteint jamais vraiment la distribution à cause du bruit dans le pas de Langevin.

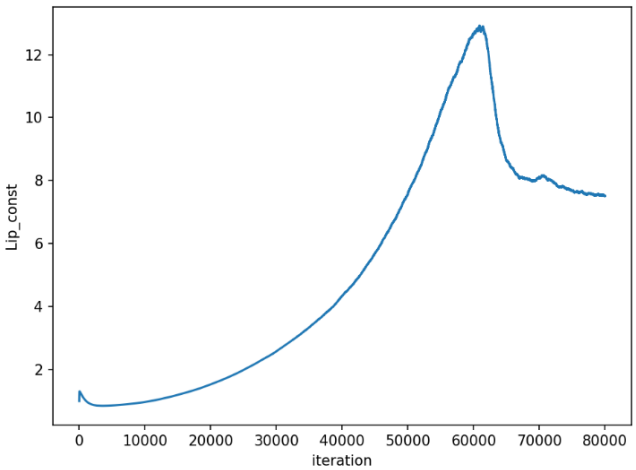
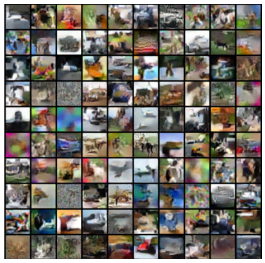


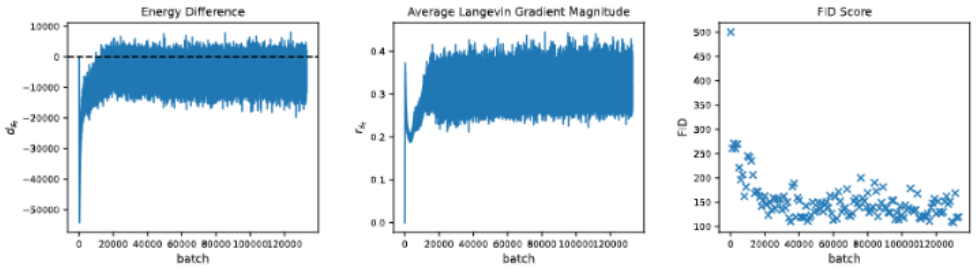
FIGURE 24 – Evolution de la constante de Lipschitz au fil des itérations sur MNIST

Il est intéressant de noter que, comme lors de la recherche manuelle de la contrainte optimale, notre heuristique a trouvé toute seule la contrainte optimale ≈ 10, ce qui renforce encore les résultats obtenus sur MNIST.

Maintenant que la théorie a été confirmée sur MNIST, nous présentons donc nos résultats sur un problème plus compliqué : CIFAR10.



(a) Buffer Lipschitz



(b) Loss de l'EBM 1-lip, gradient sur la descente de Langevin, FID d'une *long-run* = 73 (1000 steps à partir de bruit pur)

FIGURE 25 – Résultats de la méthode Lipschitz avec un K optimal sur CIFAR10

Sur CIFAR-10, les résultats restent en deçà de ceux obtenus sur MNIST. La complexité plus élevée du jeu de données, variabilité visuelle importante, images RGB, structures spatiales riches, rend le paysage énergétique bien plus difficile à couvrir avec un modèle K-Lipschitz, même adaptatif.

La figure ci-contre illustre bien la difficulté rencontrée. La constante de Lipschitz est très faible, probablement parce que la distance de Wasserstein est mal estimée. De plus, le modèle peine à traiter ce problème et guide donc mal la dynamique de Langevin vers des échantillons négatifs de qualité, ce qui entraîne une mauvaise convergence de la fonction de coût, et ainsi de suite.

Nous pouvons observer néanmoins que la constante de Lipschitz finit par se stabiliser et évolue globalement dans le sens souhaité, ce qui montre que le mécanisme de régularisation fonctionne en partie. Cependant, cette stabilisation reste insuffisante pour capturer la complexité et la diversité des distributions présentes dans CIFAR-10, ce qui explique les performances limitées du modèle sur ce jeu de données.

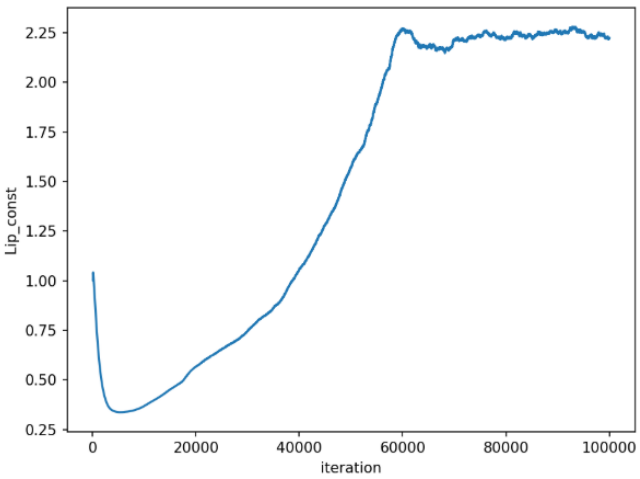
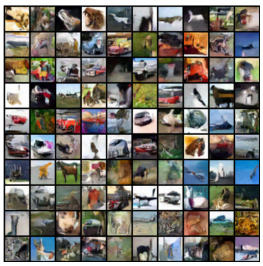


FIGURE 26 – Evolution de la constante de Lipschitz au fil des itérations sur CIFAR10

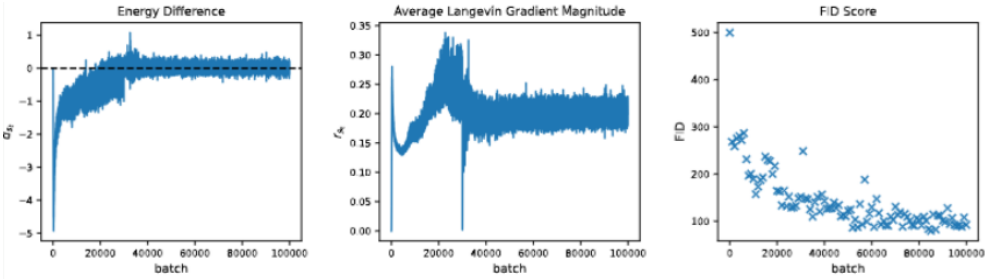
Les sous-évaluations signalées en Remarque 4 se cumulent et limitent la précision du calibrage. Ces résultats montrent les limites actuelles de l'heuristique mais motivent la recherche d'un compromis entre la stabilité apportée par la contrainte de Lipschitz et l'expressivité des EBMs non contraints avec par exemple la méthode de pré-initialisation et Semi-Lipschitz présentées ci-après.

6.4 Pré-initialisation Lipschitz

Nous montrons dans cette section les résultats obtenus avec la méthode de pré-initialisation :



(a) Buffer



(b) Loss du pipeline, gradient sur la descente de Langevin, FID d'une *long-run* = 47 (1000 steps à partir de bruit pur)

FIGURE 27 – Résultats de la méthode de pré-initialisation Lipschitz

La méthode de pré-initialisation produit des résultats qui combinent les avantages des deux approches précédentes : la stabilité du réseau Lipschitz et la qualité d'expression du modèle non contraint. Avec un FID de 47 sur le buffer, nous retrouvons pratiquement le niveau de la baseline (FID = 46), tout en conservant la robustesse caractéristique de la contrainte de Lipschitz.

Le changement de phase, passage du modèle contraint au modèle non contraint, est clairement visible dans les courbes d'apprentissage aux alentours de 40 000 itérations, soit précisément au moment où le modèle Lipschitz a atteint sa convergence. Cette transition se déroule sans aucune instabilité : aucune explosion de gradient, aucun effondrement du FID, ce qui valide l'hypothèse centrale de la pré-initialisation, le buffer pré-rempli par le modèle contraint fournit au modèle non contraint un point de départ suffisamment bien positionné pour éviter tout comportement chaotique.

La phase de raffinement se déroule ensuite très efficacement : le modèle non contraint exploite pleinement son expressivité depuis un buffer déjà de bonne qualité, et améliore régulièrement son FID. Là encore, la réduction à 10 pas de Langevin a permis d'accélérer considérablement l'entraînement par rapport à la baseline. Cette rapidité a rendu possible de pousser l'entraînement jusqu'à 100 000 itérations, et les courbes suggèrent que la véritable convergence du pipeline complet intervient aux alentours de 80 000 itérations, soit 40000 itérations encore pour le modèle non Lipschitz.

Remarque 8. *La qualité supérieure de cette méthode par rapport au réseau Lipschitz seul, à coût computationnel similaire, confirme que la contrainte de Lipschitz est plus efficace comme mécanisme d'initialisation que comme objectif final. Elle structure l'espace de recherche dans les premières phases d'entraînement là où cela est le plus nécessaire, puis laisse place à un modèle plus expressif pour capturer les détails fins de la distribution.*

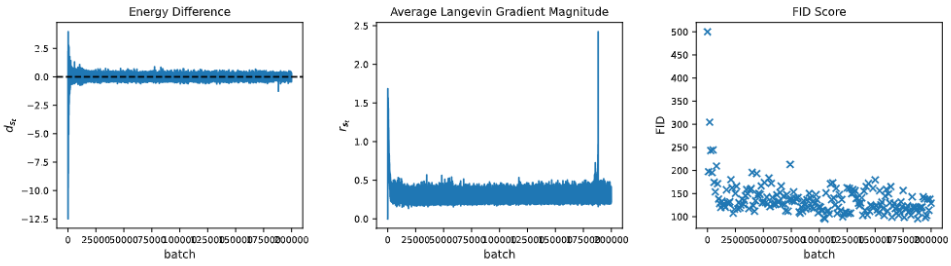
6.5 Semi-Lipschitz

Ainsi, afin de pouvoir profiter de la stabilité de la contrainte de Lipschitz et de l'expressivité des réseaux non contraints tout en conservant une architecture simple, nous proposons la méthode Semi-Lipschitz.

Ci-dessous les résultats obtenus sur CIFAR-10 :



(a) Buffer



(b) Loss du pipeline, gradient sur la descente de Langevin, FID d'une *long-run* = 51 (1000 steps à partir de bruit pur)

FIGURE 28 – Résultats de la méthode Semi-Lipschitz

Nous observons que la qualité du buffer obtenue avec la méthode Semi-Lipschitz est comparable à celle de la pré-initialisation, ce qui confirme que cette méthode permet elle aussi d'exploiter la stabilité de la contrainte de Lipschitz tout en conservant une bonne partie de l'expressivité du modèle non contraint. Le FID de 51 reste légèrement moins bon que celui de la pré-initialisation Lipschitz (FID = 47), et légèrement en retrait par rapport à la baseline (FID = 46), mais l'écart est suffisamment faible pour ne pas remettre en cause l'intérêt de l'approche.

Il est important de noter que, contrairement aux méthodes précédentes, nous ne pouvons pas ici appliquer l'heuristique de calibrage adaptatif de la constante K décrite en section 4.3. En effet, comme signalé dans la Remarque 6, la fonction de coût de la partie non contrainte du réseau ne bénéficie d'aucune borne de Lipschitz garantie, ce qui nous empêche de récupérer une estimation fiable de la distance de Wasserstein à partir de la seule fonction de coût de l'EBM. Sans cette estimation, l'heuristique adaptative perd son fondement théorique et ne peut plus piloter correctement l'évolution de K. Nous choisissons donc de fixer la contrainte de Lipschitz à $K = 1$ sur la partie contrainte du réseau, afin d'imposer une stabilité maximale et de compenser ainsi l'absence de calibrage dynamique.

Contrairement à la pré-initialisation qui nécessite de gérer deux entraînements distincts (un modèle contraint puis un modèle non contraint impose deux architectures à maintenir en mémoire), la méthode Semi-Lipschitz repose sur un seul modèle. Cette méthode permet donc :

- une architecture unique, sans logique de changement de phase ni de point de bascule à détecter.
- une empreinte mémoire réduite, puisqu'il n'est pas nécessaire de conserver deux configurations du réseau ou de dupliquer le buffer lors de la transition et un coût global réduit par rapport à la baseline (discuté en Annexe D).
- moins d'hyper-paramètres à régler. Uniquement le nombre de couches 1-Lipschitz

Remarque 9. *La méthode Semi-Lipschitz est donc un compromis entre une petite marge de qualité (FID) au profit d'une architecture plus élégante, plus simple à mettre en œuvre et moins coûteuse en ressources.*

Ce compromis qualité/simplicité conclut notre comparaison des différentes méthodes proposées, et sera repris dans la section suivante consacrée à la synthèse des résultats.

7 Conclusion

Ce rapport documente le déroulé de mon stage consacré à la stabilisation des Modèles Basés sur l'Énergie (EBM) par contrainte de Lipschitz. Nous avons exploré dans quelle mesure imposer une régularité structurelle sur la fonction d'énergie peut rendre l'entraînement des EBM plus prévisible, plus robuste et plus efficace.

Sur le plan théorique, nous avons établi un lien formel entre la contrainte de Lipschitz et la distance de Wasserstein : contraindre le réseau d'énergie à être 1-Lipschitz permet de faire le lien entre la fonction de coût contrastive d'un EBM et la dualité de Kantorovich–Rubinstein, lui conférant une interprétation géométrique claire et stable. Nous avons également montré que la contrainte Lipschitz et le mécanisme de buffer de replay sont deux mécanismes complémentaires : le buffer réduit le coût de transport effectif entre itérations, tandis que la contrainte K-Lipschitz garantit que le buffer ne devient pas obsolète trop rapidement en bornant la dérive du paysage énergétique. Cette analyse nous a permis de dériver une heuristique adaptative pour calibrer la constante K en fonction des conditions d'entraînement courantes, ainsi qu'un potentiel critère d'arrêt fondé sur la dynamique du buffer.

Sur le plan expérimental, plusieurs approches ont été comparées à une baseline non contrainte sur CIFAR-10 et MNIST. Le réseau Lipschitz pur confirme l'hypothèse centrale : la stabilité est maximale et l'entraînement n'a jamais divergé, même avec dix fois moins de pas de Langevin. Ce gain de robustesse a pour contrepartie un FID dégradé, reflet du compromis expressivité/stabilité inhérent à la contrainte. La méthode de pré-initialisation résout ce compromis en découplant les deux phases : le modèle contraint initialise un buffer de qualité, que le modèle non contraint affine ensuite librement. Nous retrouvons ainsi un FID équivalent à la baseline, avec la stabilité du modèle Lipschitz et un entraînement dix fois plus rapide.

De plus, en imposant d'abord une régularité Lipschitzienne, nous avons montré que cela construit un paysage énergétique stable et bien structuré, dans lequel un réseau non contraint peut ensuite exprimer librement sa capacité à capturer les détails fins de la distribution, sans risquer de faire diverger l'entraînement. Cette idée peut se décliner de deux manières : soit séquentiellement, comme dans la pré-initialisation, où un modèle Lipschitz entraîne d'abord un buffer stable avant de laisser place à un modèle non contraint ; soit conjointement au sein d'une seule architecture, comme dans la méthode Semi-Lipschitz, où un extracteur de caractéristiques Lipschitz façonne un espace stable sur lequel vient s'appuyer une tête non contrainte, plus expressive. Le Semi-Lipschitz confirme cependant qu'un compromis reste accessible entre le FID et la simplicité avec une architecture unique, plus simple et moins coûteuse en mémoire. (Un tableau bilan est donné en Annexe F afin de discuter des différentes pistes)

Plus généralement, ce stage illustre comment deux outils isolés comme les EBMs et les réseaux Lipschitz peuvent être utilisés conjointement afin d'obtenir des résultats nouveaux et intéressants poussant encore l'importance capitale d'une Recherche mettant en avant la collaboration.

De plus, pour faire le lien avec l'introduction, nous avons fourni des pistes pour améliorer et faciliter l'utilisation des EBMs. Ces mêmes EBMs qui à terme pourraient servir à mieux comprendre le cerveau et peut-être aussi constituer une brique vers des systèmes d'apprentissage continu : des modèles suffisamment robustes et modulaires pour être combinés entre eux, capitalisant sur les connaissances déjà acquises sans jamais avoir à les réapprendre, et réduisant ainsi drastiquement le coût computationnel, associé à chaque nouveau cycle d'entraînement.

Références

[1] Cem Anil, James Lucas, and Roger Grosse. Sorting out lipschitz function approximation. In *International conference on machine learning*, pages 291–301. PMLR, 2019.

[2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. Pmlr, 2017.

[3] Michal Balcerak, Tamaz Amiranashvili, Antonio Terpin, Suprosanna Shit, Lea Bogensperger, Sebastian Kaltenbach, Petros Koumoutsakos, and Bjoern Menze. Energy matching : unifying flow matching and energy-based models for generative modeling. *Advances in neural information processing systems (NeurIPS)*, 2025.

[4] Louis Béthune, Thibaut Boissin, Mathieu Serrurier, Franck Mamalet, Corentin Friedrich, and Alberto Gonzalez Sanz. Pay attention to your loss : understanding misconceptions about lipschitz neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 35 :20077–20091, 2022.

[5] Louis Béthune, Paul Novello, Thibaut Boissin, Guillaume Coiffier, Mathieu Serrurier, Quentin Vincenot, and Andres Troya-Galvis. Robust one-class classification with signed distance function using 1-lipschitz neural networks. *arXiv preprint arXiv :2303.01978*, 2023.

[6] Åke Björck and Clazett Bowie. An iterative algorithm for computing the best estimate of an orthogonal matrix. *SIAM Journal on Numerical Analysis*, 8(2) :358–364, 1971.

[7] Fabio Brau, Giulio Rossolini, Alessandro Biondi, and Giorgio Buttazzo. Robust-by-design classification via unitary-gradient neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 14729–14737, 2023.

[8] Louis Béthune, Thibaut Boissin, Mathieu Serrurier, Franck Mamalet, Corentin Friedrich, and Alberto González-Sanz. Pay attention to your loss : understanding misconceptions about 1-lipschitz neural networks, 2022.

[9] Jiequan Cui, Zhuotao Tian, Zhisheng Zhong, Xiaojuan Qi, Bei Yu, and Hanwang Zhang. Decoupled kullback-leibler divergence loss. *Advances in Neural Information Processing Systems (NeurIPS)*, 37 :74461–74486, 2024.

[10] Yilun Du and Igor Mordatch. Implicit generation and modeling with energy based models. *Advances in neural information processing systems (NeurIPS)*, 32, 2019.

[11] Nicholas V Findler. A model-based theory for déjà vu and related psychological phenomena. *Computers in Human Behavior*, 14(2) :287–301, 1998.

[12] Chelsea Finn, Paul Christiano, Pieter Abbeel, and Sergey Levine. A connection between generative adversarial networks, inverse reinforcement learning, and energy-based models. *arXiv preprint arXiv :1611.03852*, 2016.

[13] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boisbunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T.H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. Pot : Python optimal transport. *Journal of Machine Learning Research*, 22(78) :1–8, 2021.

[14] Will Grathwohl, Jacob Kelly, Milad Hashemi, Mohammad Norouzi, Kevin Swersky, and David Duvenaud. No mcmc for me : Amortized sampling for fast and stable training of energy-based models. *arXiv preprint arXiv :2010.04230*, 2020.

[15] Will Grathwohl, Kuan-Chieh Wang, Jörn-Henrik Jacobsen, David Duvenaud, Mohammad Norouzi, and Kevin Swersky. Your classifier is secretly an energy based model and you should treat it like one. *arXiv preprint arXiv :1912.03263*, 2019.

[16] Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pages 1352–1361. PMLR, 2017.

[17] Geoffrey E. Hinton. Products of experts. In *Proceedings of the Ninth International Conference on Artificial Neural Networks*, 1999.

[18] Geoffrey E Hinton. Training products of experts by minimizing contrastive divergence. *Neural computation*, 14(8) :1771–1800, 2002.

[19] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8) :1771–1800, 2002.

[20] Georgi Hrusanov, Oliver Y Chén, and Julien S Bodelet. Generative model via quantile assignment. *arXiv preprint arXiv :2602.18216*, 2026.

[21] Sadeep Jayasumana, Srikumar Ramalingam, Andreas Veit, Daniel Glasner, Ayan Chakrabarti, and Sanjiv Kumar. Rethinking fid : Towards a better evaluation metric for image generation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9307–9315, 2024.

[22] Taesup Kim and Yoshua Bengio. Deep directed generative models with energy-based probability estimation. *arXiv preprint arXiv :1606.03439*, 2016.

[23] Rithesh Kumar, Sherjil Ozair, Anirudh Goyal, Aaron Courville, and Yoshua Bengio. Maximum entropy generators for energy-based models. *arXiv preprint arXiv :1901.08508*, 2019.

[24] Marjan Laal. Lifelong learning : What does it mean ? *Procedia - Social and Behavioral Sciences*, 28 :470–474, 2011. World Conference on Educational Technology Researches - 2011.

[25] Yann LeCun, Sumit Chopra, Raia Hadsell, M Ranzato, Fujie Huang, et al. A tutorial on energy-based learning. *Predicting structured data*, 1(0), 2006.

[26] Bojie Li. Incompressible knowledge probes : Estimating black-box llm parameter counts via factual capacity. *arXiv preprint arXiv :2604.24827*, 2026.

[27] Chuan Li. Openai's gpt-3 language model : A technical overview. *Lambda AI*, 2020.

[28] Yingzhen Li, Richard E Turner, and Qiang Liu. Approximate inference with amortised mcmc. *arXiv preprint arXiv :1702.08343*, 2017.

[29] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. *arXiv preprint arXiv :1802.05957*, 2018.

[30] Andriy Mnih and Geoffrey Hinton. Learning nonlinear constraints with contrastive backpropagation. In *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.*, volume 2, pages 1302–1307. IEEE, 2005.

[31] Arthur N. Montanari, Francesco Bullo, Dmitry Krotov, and Adilson E. Motter. Energy-based dynamical models for neurocomputation, learning, and optimization, 2026.

[32] Igor Mordatch. Concept learning with energy-based models. *Workshop ICLR*, 2018.

[33] Erik Nijkamp, Mitch Hill, Tian Han, Song-Chun Zhu, and Ying Nian Wu. On the anatomy of mcmc-based maximum likelihood learning of energy-based models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04) :5272–5280, Apr. 2020.

[34] Erik Nijkamp, Mitch Hill, Song-Chun Zhu, and Ying Nian Wu. Learning non-convergent non-persistent short-run mcmc toward energy-based model. *Advances in Neural Information Processing Systems (NeurIPS)*, 32, 2019.

[35] Ning Ning. Metropolis-adjusted subdifferential langevin algorithm. *arXiv preprint arXiv :2507.06950*, 2025.

[36] G. Parisi. Correlation functions and computer simulations. *Nuclear Physics B*, 180(3) :378–384, 1981.

[37] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv :2104.10350*, 2021.

[38] European Environment Agency Press. Average co2 emissions from new cars and new vans slightly increased in 2024. *European Environment Agency*, 2025.

[39] Ruslan Salakhutdinov and Geoffrey Hinton. Deep boltzmann machines. In David van Dyk and Max Welling, editors, *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, volume 5 of *Proceedings of Machine Learning Research*, pages 448–455, Hilton Clearwater Beach Resort, Clearwater Beach, Florida USA, 16–18 Apr 2009. PMLR.

[40] Mathieu Serrurier, Franck Mamalet, Alberto González-Sanz, Thibaut Boissin, Jean-Michel Loubes, and Eustasio Del Barrio. Achieving robustness in classification using optimal transport with hinge regularization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 505–514, 2021.

[41] H.J. Sussmann. Learning algorithms for boltzmann machines. In *Proceedings of the 27th IEEE Conference on Decision and Control*, pages 786–791 vol.1, 1988.

[42] Tijmen Tieleman. Training restricted boltzmann machines using approximations to the likelihood gradient. In *Proceedings of the 25th international conference on Machine learning*, pages 1064–1071, 2008.

[43] Richard Turner. Cd notes. 2005.

[44] Cédric Villani. *The Wasserstein distances*, pages 93–111. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.

[45] Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning : Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(8) :5362–5383, 2024.

[46] Jianwen Xie, Yang Lu, Ruiqi Gao, and Ying Nian Wu. Cooperative learning of energy-based model and latent variable model via mcmc teaching. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[47] Jianwen Xie, Zilong Zheng, and Ping Li. Learning energy-based model with variational auto-encoder as amortized sampler. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(12) :10441–10451, May 2021.

[48] Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. Model merging in llms, mllms, and beyond : Methods, theories, applications, and opportunities. *ACM Comput. Surv.*, 58(8), February 2026.

[49] Xuwang Yin, Claire Zhang, Julie Steele, Nir Shavit, and Tony T Wang. Joint discriminative-generative modeling via dual adversarial training. *arXiv preprint arXiv :2510.13872*, 2025.

[50] Peiyu Yu, Yaxuan Zhu, Sirui Xie, Xiaojian Ma, Ruiqi Gao, Song-Chun Zhu, and Ying Nian Wu. Learning energy-based prior model with diffusion-amortized mcmc, 2023.

[51] Yang Yu, Jiahui Wang, Yu Liu, Pingfeng Yu, Dongsheng Wang, Ping Zheng, and Meng Zhang. Revisit the environmental impact of artificial intelligence : the overlooked carbon emission source ? *Frontiers of Environmental Science & Engineering*, 18(12) :158, 2024.

[52] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks, 2017.

[53] Shuangfei Zhai, Yu Cheng, Weining Lu, and Zhongfei Zhang. Deep structured energy based models for anomaly detection. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1100–1109, New York, New York, USA, 20–22 Jun 2016. PMLR.

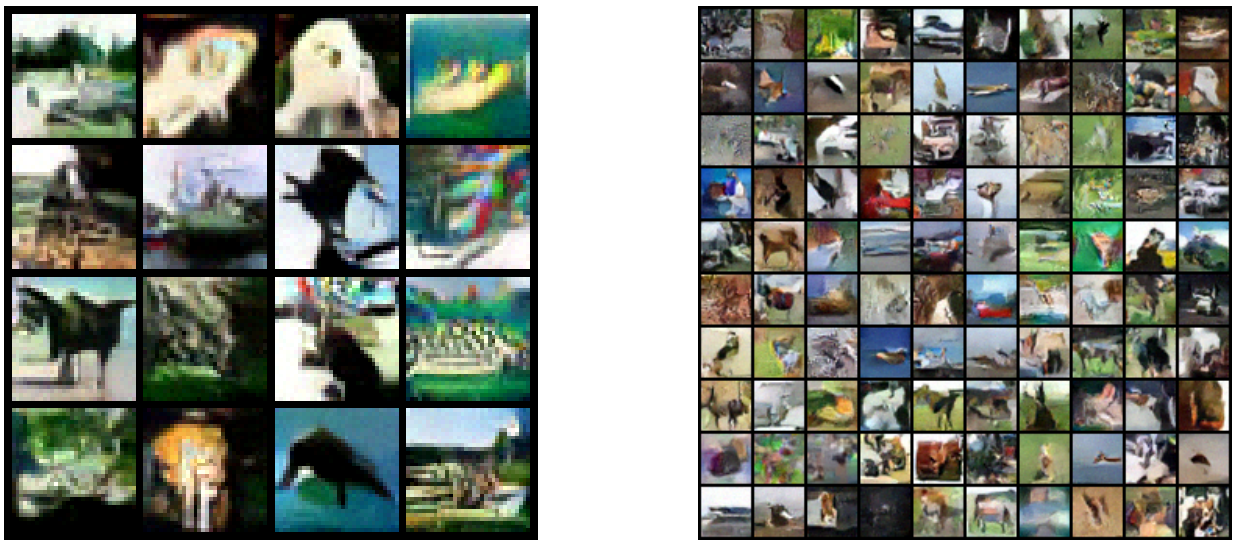
[54] Junbo Zhao, Michael Mathieu, and Yann LeCun. Energy-based generative adversarial network. *arXiv preprint arXiv :1609.03126*, 2016.

[55] Yang Zhao, Jianwen Xie, and Ping Li. Learning energy-based generative models via coarse-to-fine expanding and sampling. In *International Conference on Learning Representations*, 2020.

[56] Song Zhu, Yingnian Wu, and David Mumford. Filter, random field and maximum entropy. *Neural Computation - NECO*, 01 1997.

A Long-run et Buffer

Au cours de nos expériences, nous avons observé un écart systématique entre la qualité et la diversité des images issues du **buffer** et celle des images issues d'une **long-run**. Le buffer est un artefact direct de l'entraînement, tandis que la long-run part de bruit pur et doit parcourir l'intégralité du paysage énergétique (cf figure 29).



(a) Long-run (b) Buffer

FIGURE 29 – Exemple d'une *long-run* d'un modèle versus le buffer associé à l'entraînement de ce même modèle.

Cette dernière est donc souvent de moins bonne qualité, et fréquemment *non convergente* au sens de Nijkamp et al. [34]. Si la dynamique de Langevin fait baisser le FID du lot d'images sur lequel nous réalisons l'opération puis le fait réaugmenter après *trop* de pas de Langevin, alors le paysage énergétique appris n'est pas représentatif du jeu de données cible et l'apprentissage s'est mal passé (voir Figure 30).

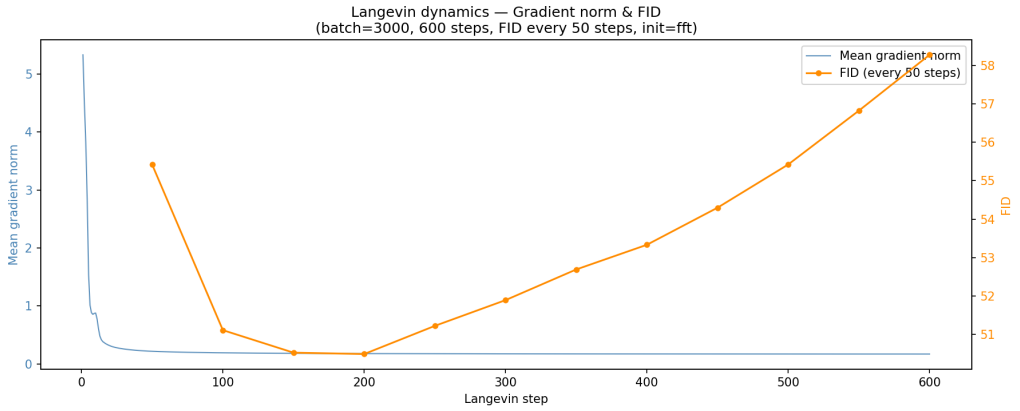


FIGURE 30 – FID d'une *long-run* en fonction du nombre de pas de Langevin réalisés. Nous observons donc la non-convergence du modèle : au-delà d'un certain seuil, ajouter des pas de Langevin dégrade le FID et éloigne l'image générée de la distribution cible. Ce seuil semble correspondre au nombre moyen de pas m_n vus par le buffer (cf. Eq. 23), ce qui suggère une forme de *sur-apprentissage* de l'EBM vis-à-vis du nombre de pas qu'il observe pendant l'entraînement.

B Régularisation

Dans cette section nous discutons de la régularisation L2 applicable dans la fonction de coût d'un EBM. Cette dernière permet de forcer les énergies du modèle à se rapprocher de 0 :

$$L_2^{Rgularisation} = \mathbb{E}\left[E_\theta(x^+)^2\right] + \mathbb{E}\left[E_\theta(x^-)^2\right]$$

Le gradient de la fonction de coût régularisée s'écrit alors :

$$\nabla_\theta \mathcal{L}_{reg} = \nabla_\theta \mathcal{L}_{EBM} + 2\,\nabla_\theta E_\theta(x^+) \cdot E_\theta(x^+) + 2\,\nabla_\theta E_\theta(x^-) \cdot E_\theta(x^-) \tag{26}$$

où $\mathcal{L}_{EBM}$ est la fonction de coût contrastive définie en Eq. 14.

Idée clé

Ce terme additionnel pousse les valeurs d'énergie $E_\theta(x^+)$ et $E_\theta(x^-)$ à se rapprocher de 0, réduisant ainsi leur amplitude. Cela se traduit par des gradients moins susceptibles d'exploser : sans régularisation, l'énergie peut varier sur une plage très large (par exemple de 0 à -2000), tandis qu'avec régularisation cette plage devient beaucoup plus restreinte (par exemple de 0 à -10).

Cette régularisation permet effectivement de stabiliser l'entraînement et peut permettre d'aider pour résoudre ce problème de *non-convergence* des long-run, mais au prix d'un compromis sur la qualité du buffer : lorsque la régularisation est trop forte, le buffer devient très *sparse* (voir figure 31), c'est-à-dire composé d'un grand nombre d'échantillons encore proches du bruit et se raffinant lentement, pour seulement une faible proportion d'images de bonne qualité. Tout se passe comme si la régularisation ralentissait effectivement le pas de Langevin.



FIGURE 31 – Buffer avec Régularisation

Nous faisons l'hypothèse que le paysage énergétique, contraint à conserver de faibles valeurs par la régularisation, tend également à voir la force de ses variations réduite, et donc à produire des normes de gradient $\nabla_x E_\theta(x)$ plus faibles. Un gradient de plus faible amplitude guide alors plus lentement la dynamique de Langevin, ce qui aurait pour effet de la *ralentir*. Cette hypothèse reste toutefois incertaine : rien ne garantit formellement qu'une régularisation sur l'amplitude de l'énergie entraîne mécaniquement une réduction de la norme de son gradient par rapport à x.

C DualHead : Architecture propre pour la pré-initialisation Lipschitz et HKR

Le pipeline à pré-initialisation Lipschitz présente de meilleures performances que le Semi-Lipschitz. Nous présentons donc dans cette section les travaux que nous avons réalisés pour tenter de profiter de ces performances tout en construisant une architecture plus élégante. En effet, dans la pré-initialisation, nous maintenons deux réseaux séparés, entraînés séquentiellement, et le modèle final $E_{\phi,\theta}$ n'est pas un EBM au sens strict. Nous proposons ici une architecture unifiée qui cherche à résoudre ces limitations.

C.1 Description de l'architecture dual-head.

L'idée est de remplacer les deux modèles par un seul réseau à deux têtes :

— un **corps partagé** f_ψ, extracteur de caractéristiques, contraint à être 1-Lipschitz,

— une **tête HKR** h_θ^{Lip}, linéaire et Lipschitz, entraînée avec la fonction de coût HKR,

— une **tête EBM** h_ϕ^{EBM}, linéaire sans contrainte, entraînée avec la fonction de coût EBM classique.

Le corps f_ψ étant 1-Lipschitz, la composition $h_\theta^{\text{Lip}} \circ f_\psi$ l'est également, ce qui garantit les propriétés de la pré-initialisation vues précédemment. La tête EBM, elle, bénéficie de la richesse des caractéristiques extraites par f_ψ sans être bridée par la contrainte.

C.2 La fonction de coût HKR.

Jusqu'ici, le modèle Lipschitz était entraîné sur la fonction de coût classique d'un EBM qui est une approximation de distance de Wasserstein entre p_θ et p_D lorsque le réseau est 1-Lipschitz (cf. 22). Nous introduisons maintenant la fonction de coût **HKR (Hinge-Kantorovich-Rubinstein)** [40], une variante qui centre les sorties autour de 0, dans notre cas :

$$\mathcal{L}_{\text{HKR}}(f) = \mathcal{L}_{\text{EBM}-1\text{lip}}(f) + \mathbb{E}_{x\sim p_D \cup p_\theta}\left[\max\left(0,\, \frac{m}{2} \pm f(x)\right)\right] \tag{27}$$

où $m > 0$ est une marge, si les éléments appartiennent à la distribution cible ($x \sim p_D$) alors nous cherchons à obtenir une sortie négative ($f(x) < 0$) avec une marge minimale de m par rapport à 0 ($\max\left(0,\, \frac{m}{2} + f(x)\right)$). Et inversement pour les échantillons de la distribution p_θ.

La propriété clé est la suivante : après entraînement, les sorties de la tête HKR sont **positives pour les échantillons de bruit** (ou difficiles à traiter) et **négatives pour les échantillons proches de la distribution cible**. Le zéro joue donc le rôle de frontière naturelle entre les deux régimes à l'image d'un classifieur [5].

C.3 Inférence conjointe par pondération sigmoïde

Nous disposons donc de deux énergies complémentaires :

— $E_{\text{HKR}}(x)$: **fiable** partout, y compris dans les zones de bruit, mais moins expressive,

— $E_{\text{EBM}}(x)$: très **expressive** près de la distribution cible, mais très **instable** dans les zones de bruit.

Pour combiner les deux, nous utilisons la sortie de la tête HKR comme **signal de confiance** via une sigmoïde (considérée comme constante dans le calcul de la fonction de coût) :

$$\alpha(x) = \sigma(E_{\text{HKR}}(x))\,, \qquad E_{\theta,\psi,\phi}(x) = \alpha(x)\,E_{\text{HKR}}(x) + (1-\alpha(x))\,E_{\text{EBM}}(x) \tag{28}$$

L'intuition est simple :

— Si $E_{\text{HKR}}(x) \gg 0$ (bruit pur), alors $\alpha \approx 1$ et $E_\theta \approx E_{\text{HKR}}$: nous nous appuyons sur le modèle Lipschitz, le seul étant robuste sur ces régions.

— Si $E_{\text{HKR}}(x) \ll 0$ (proche des données), alors $\alpha \approx 0$ et $E_\theta \approx E_{\text{EBM}}$: nous basculons sur la tête non contrainte, plus expressive, là où elle est fiable.

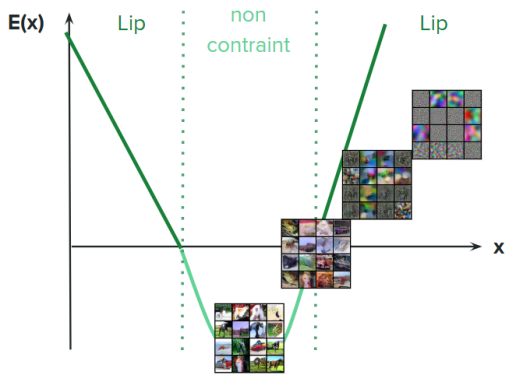


FIGURE 32 – Paysage énergétique d'un pipeline dual-head

La sigmoïde assure une transition douce entre les deux régimes, sans **discontinuité**. Cette architecture permet ainsi d'entraîner un seul modèle, de conserver un modèle proche d'un EBM (le modèle final $E_{\theta,\psi,\phi}$ définit bien une fonction d'énergie sur tout l'espace), et d'éliminer le besoin de maintenir deux réseaux séparés en mémoire.

De plus, la génération d'éléments négatifs x^- est beaucoup plus simple, contrairement à la méthode précédente où il fallait d'abord faire un certain nombre de pas avec E_θ^{Lip} puis les pas de Langevin de E_ϕ, dans notre cas il suffit de faire une seule descente de gradient sur $E_{\theta,\psi,\phi}$ avec un petit nombre de pas. (cf algorithme 3)

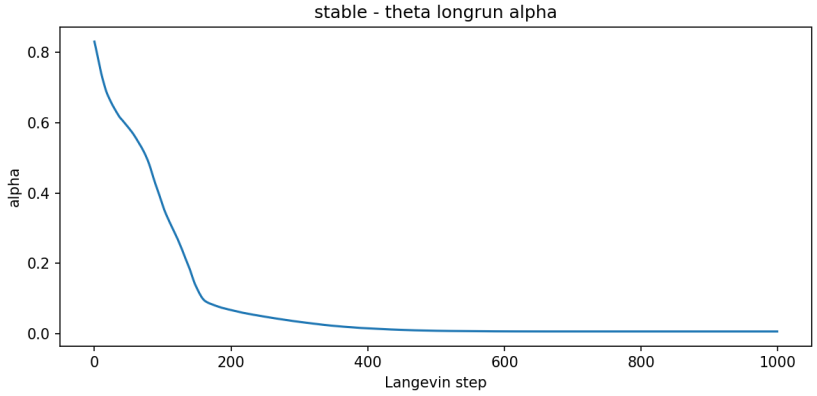
Algorithm 3 Entraînement de l'architecture Dual-Head HKR-EBM

Require: Jeu de données $\mathcal{D}$, modèle dual-head $E_{\phi,\theta}$, ϵ, nombre de pas MCMC λ

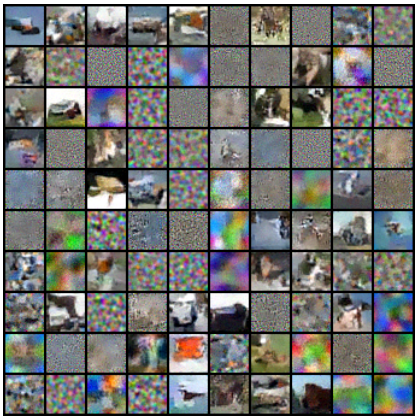
Instancier un buffer $\mathcal{B} \leftarrow \mathcal{N}(\mathbf{0}, \mathbf{I})$
for chaque itération d'entraînement **do**
  Échantillonnage d'images réelles : $\{\mathbf{x}^+\} \sim p_{\mathcal{D}}$
  Initialiser les échantillons négatifs : $\mathbf{x}_0^- \sim \mathcal{B}$ pour 95% des cas, $\mathcal{N}(\mathbf{0}, \mathbf{I})$ sinon
  for $t = 1, \ldots, \lambda$ **do**
    Mettre à jour : $\mathbf{x}_t^- = \mathbf{x}_{t-1}^- - \frac{\epsilon}{2}\nabla_{\mathbf{x}} E_{\phi,\theta}(\mathbf{x}_{t-1}^-) + \sqrt{\epsilon}\boldsymbol{\eta}_t$
  end for
  — **Mise à jour de la tête HKR** —
  Calculer la fonction de coût HKR définie par l'équation (27)
  — **Mise à jour de la tête EBM** —
  Calculer la fonction de coût EBM définie par l'équation (22)
  Mise à jour des poids avec $Loss = L_{HKR} + L_{EBM}$
  $\mathcal{B} \leftarrow \mathbf{x}_t^-$
end for

C.4 Résultats

Cette méthode semble théoriquement fonctionner mais en pratique présente de mauvais résultats.



(a) Valeur de α en fonction du nombre de pas de Langevin fait sur une *long-run*



(b) Buffer

FIGURE 33 – Résultats de la méthode Dual-Head

C.4.1 Nécessité d'un rééquilibrage des gradients

Un premier problème apparaît directement au niveau du corps partagé f_ψ : la tête HKR n'arrive pas à faire correctement la distinction entre bruit et données si nous laissons la fonction de coût EBM contribuer telle quelle au gradient de ψ. En effet, les deux têtes partagent le même extracteur de caractéristiques, mais rien ne garantit que leurs gradients respectifs aient la même échelle. Dans les faits, le gradient de $\mathcal{L}_{\text{EBM}}$ par rapport à ψ domine largement celui de $\mathcal{L}_{\text{HKR}}$, si bien que le corps partagé se spécialise pour la tête non contrainte au détriment de la tête Lipschitz.

Pour corriger ce déséquilibre, il est possible de re-normaliser la contribution de la perte EBM par le rapport des normes des gradients des deux pertes par rapport aux paramètres du corps partagé, de sorte que les deux têtes fassent évoluer f_ψ à la même vitesse :

$$Loss = \mathcal{L}_{\text{HKR}} + \frac{\|\nabla_\psi \mathcal{L}_{\text{HKR}}\|}{\|\nabla_\psi \mathcal{L}_{\text{EBM}}\|} \mathcal{L}_{\text{EBM}} \tag{29}$$

Ce facteur, recalculé à chaque itération, agit comme un poids adaptatif qui force la tête EBM à contribuer au corps partagé dans les mêmes proportions que la tête HKR, sans quoi le corps f_ψ dérive et la tête HKR perd sa capacité à distinguer proprement le bruit des données.

C.4.2 Nécessité d'une régularisation de la tête EBM

Ce rééquilibrage des gradients ne suffit cependant pas à lui seul. Il faut en plus régulariser la sortie de la tête EBM (cf. Annexe B), sans quoi ses valeurs s'éloignent arbitrairement de 0 au cours de l'entraînement. Or, la sortie de la tête HKR est bornée par construction dans $\left[-\frac{m}{2}, \frac{m}{2}\right]$ du fait de la marge m de la fonction de coût HKR, ce qui donne dans notre cas des valeurs typiquement comprises entre -0.5 et 0.5. La tête EBM, elle, n'étant soumise à aucune contrainte de ce type, peut produire des valeurs beaucoup plus basses ou beaucoup plus hautes.

Cette différence d'échelle est problématique pour la pondération sigmoïde $\alpha(x) = \sigma(E_{\text{HKR}}(x))$: celle-ci n'a de sens que si les deux énergies évoluent dans des gammes de valeurs comparables. Si l'énergie EBM diverge, la combinaison $\alpha(x)\, E_{\text{HKR}}(x) + (1 - \alpha(x))\, E_{\text{EBM}}(x)$ devient incohérente : le poids α, calculé uniquement à partir de la sortie HKR (bornée), ne reflète plus l'incertitude réelle vis-à-vis de valeurs EBM potentiellement très extrêmes.

C.4.3 Comportement du buffer et compromis vitesse/qualité

Ces deux correctifs, une fois combinés, changent qualitativement la dynamique d'entraînement observée dans le buffer. Comme le montre la figure 33b, le buffer se raffine plutôt mal et lentement (comme expliqué dans l'Annexe B : les échantillons négatifs restent longtemps dans un régime dominé par la tête HKR avant de commencer à ressembler à la distribution cible).

La figure 33a confirme cependant que le flux algorithmique fonctionne comme voulu : au début des pas de Langevin, α vaut environ 1, ce qui signifie que l'énergie utilisée est presque exclusivement celle de la tête HKR ($E_\theta \approx E_{\text{HKR}}$). Puis, à mesure que les échantillons négatifs se rapprochent de la distribution cible au fil des pas, α décroît progressivement, laissant la tête EBM prendre le relais de manière graduelle et sans discontinuité.

Le comportement observé est donc qualitativement correct : la transition HKR $\rightarrow$ EBM se fait bien, progressivement, et le rôle de garde-fou de la tête HKR est respecté. Cependant, ce compromis se traduit en pratique par un FID plutôt mauvais ($FID \approx 80$), la lenteur du raffinement du buffer et la complexité du système limitant la qualité finale des échantillons générés dans le temps d'entraînement imparti.

D Économie computationnelle

Dans cette section nous étudions l'économie computationnelle potentielle de notre dernière méthode Semi-Lipschitz.

Le principal coût computationnel d'un EBM provient de la génération des échantillons négatifs par dynamique de Langevin. À chaque itération d'entraînement, plusieurs pas de MCMC doivent être réalisés afin d'obtenir des échantillons suffisamment représentatifs de la distribution modélisée. Cette phase est généralement bien plus coûteuse que la mise à jour des paramètres du réseau elle-même.

Deuxièmement, un modèle non contraint doit avoir en moyenne une centaine de pas de Langevin pour fonctionner correctement. Avec un modèle Lipschitz dans le pipeline, il est possible de garantir une stabilité et une expressivité avec une dizaine de pas de Langevin soit dix fois moins ! Ce qui accélère grandement l'entraînement et la génération.

Exemple

Prenons un modèle non contraint qui s'entraîne sur 100 pas de Langevin par itération, une batchsize de 100 sur 40 000 itérations :
$\longrightarrow$ Nombre de pas de Langevin réalisés $= 100 * 100 * 40000 = 4 \cdot 10^8$ pas

Pour notre modèle Semi-Lipschitz, nous avons 10 fois moins de pas de Langevin et les opérations réalisées en plus sont minimes (les inférences sur les 2 têtes sont rapides car les têtes sont petites, les calculs intermédiaires (sigmoïde, etc...) ne sont pas très coûteux), ainsi :
$\longrightarrow$ Nombre de pas de Langevin réalisés $= 10 * 100 * 40000 = 4 \cdot 10^7$ pas

En prenant en compte le facteur de stabilité qui peut même avec 100 pas de Langevin faire diverger l'entraînement et devoir tout relancer. Ce qui fait des réseaux Lipschitz une très bonne piste pour les EBMs.

E Hyper-paramètres

Dans cette section nous discutons de certains hyper-paramètres utilisés et donnons notre expérience sur leur utilisation.

Le **critère d'arrêt** introduit en section 4.3 s'avère visuellement correct dans la majorité des cas. Nous observons cependant que, pour certaines configurations, le FID continue de s'améliorer légèrement après le palier m_n. Ce dépassement ne remet pas en cause l'utilité du critère : il peut dans ce cas servir de **diagnostic complémentaire**. En effet, si à l'itération m_n le buffer reste de mauvaise qualité, cela indique que l'entraînement lui-même est probablement déficient.

Le **taux de réinitialisation** r du buffer (cf. section 2.2.4) joue aussi un rôle important :

— un r plus faible permet à chaque échantillon du buffer de voir en moyenne davantage de pas de Langevin avant réinitialisation (cf. Eq. 23), ce qui améliore la qualité du buffer ;

— en contrepartie, cela réduit la fréquence à laquelle le modèle est confronté à du bruit pur pendant l'entraînement, ce qui dégrade sa capacité à bien gérer les long-runs, où le point de départ est justement du bruit.

Un taux $r = 5\%$ constitue, dans nos expériences, un bon compromis entre ces deux exigences.

Le **bruit utilisé pour initialiser le buffer** peut être un simple bruit uniforme, principalement composé de hautes fréquences. Nous avons cependant observé un gain de performance notable en initialisant le buffer avec un bruit aux fréquences plus variées, obtenu en filtrant le bruit par des coupe-bandes appliqués dans le domaine de Fourier. En effet, des jeux de données comme CIFAR-10 sont eux-mêmes composés d'une majorité de basses fréquences ; proposer à l'EBM un bruit initial plus riche en basses et moyennes fréquences permet donc de stabiliser l'entraînement en le rapprochant davantage de la structure des données réelles, tout en conservant une bonne compréhension des composantes hautes fréquences.

Nous avons également observé que, même pour un modèle non contraint, ce bruit multi-fréquence permet de réduire le nombre de pas de Langevin nécessaires à l'entraînement tout en conservant une bonne stabilité. Pour réduire encore davantage ce nombre de pas, les meilleurs résultats ont été obtenus en introduisant progressivement les fréquences plutôt qu'en les fixant :

en initialisant d'abord le buffer avec des basses fréquences, puis en augmentant graduellement la part de moyennes et hautes fréquences au fil de l'entraînement (via un *scheduling* de la fréquence du bruit injecté lors des 5% de réinitialisation). Ce résultat, intéressant en soi, confirme l'importance du point d'initialisation dans la dynamique de Langevin. Par souci de simplicité, nous avons cependant choisi de conserver dans nos expériences un bruit multi-fréquence fixe, réparti uniformément en un tiers basses, un tiers moyennes et un tiers hautes fréquences.

Concernant l'**optimiseur**, **Adam** semble être un bon candidat pour les réseaux Lipschitz, aucun test n'a été réalisé avec d'autres optimiseurs comme AdamW ou Muon.

F Bilan

Nous donnons ici un tableau des pistes les plus intéressantes et celles qui n'ont pas fonctionné pendant nos tests.

TABLE 2 – Bilan synthétique des différentes pistes explorées.

Piste	**Observations**
Baseline non contrainte (100 pas)	Fonctionne, mais entraînement long, instable et très sensible aux hyper-paramètres.
1-Lipschitz (10 pas)	Meilleure stabilité, jamais de divergence et plus rapide. Mais paysage trop lisse → perte d'expressivité, FID dégradé.
K-Lipschitz (K=10, 100)	Permet de régler manuellement le rapport expressivité/stabilité. K devient un hyper-paramètre à part entière, pas satisfaisant à régler à la main.
K optimal (heuristique adaptative)	Très bon sur MNIST (retrouve K≈10 tout seul, plus besoin de le régler). Mais pas scalable : sur CIFAR10 la distance de Wasserstein estimée est trop peu fiable (sous-évaluation), donc l'heuristique perd en précision.
Pré-initialisation Lipschitz (double réseau)	Le plus intéressant en pratique : FID ≈ baseline, stabilité du Lipschitz, transition sans aucune instabilité. Contrepartie : deux réseaux à entraîner/maintenir, le modèle final n'est plus un vrai EBM.
Semi-Lipschitz (extracteur 1-lip + tête libre)	Un seul réseau, simple, moins cher en mémoire, peu d'hyper-paramètres. FID légèrement en dessous de la pré-init, mais l'écart reste faible. Bon compromis simplicité/-qualité. Revient à devoir régler manuellement le nombre de couches Lipschitz désirées.
Dual-Head (tête HKR + tête EBM)	Mauvais en pratique (FID ≈ 80). Semble nécessiter un rééquilibrage des gradients entre les deux têtes + régularisation de la tête EBM. Buffer se raffine lentement.
Régularisation L2 de l'énergie	Stabilise un peu (réduit l'amplitude de l'énergie, évite l'explosion des gradients), mais peut brider l'expressivité : buffer devient sparse si la régularisation est trop forte, ralentit le Langevin.

suite page suivante

Table 2 – suite de la page précédente

Piste	**Résultat / Verdict**
Bruit multi-fréquence pour initialisation du buffer	Gain notable en stabilité, même pour un modèle non contraint. Introduire les fréquences progressivement marche encore mieux mais complexifie le pipeline. Très bon gain d'expressivité sur les réseaux Lipschitz. Sur le réseau Semi-Lipschitz, il est tout de même possible d'entraîner le modèle uniquement sur du bruit uniforme, il faut cependant baisser le learning rate (ex. $6e-6$ sur CIFAR10)
Taux de réinitialisation r du buffer	$r = 5\%$ = bon compromis. Trop bas $\rightarrow$ le modèle voit trop peu de bruit pur pendant l'entraînement et gère mal les long-run. Trop haut $\rightarrow$ buffer jamais assez raffiné.
Taille de l'extracteur de caractéristiques	Nous avons observé que la taille de l'extracteur de caractéristiques est très importante dans la qualité de l'apprentissage.
MALA (Langevin ajusté Metropolis-Hastings)	Corrige le biais théorique de l'ULA, mais double le coût par pas et rajoute de la variance. Non retenu car très cher pour le gain apporté.
Critère d'arrêt basé sur m_n	Visuellement correct dans la majorité des cas. Parfois le FID continue de s'améliorer un peu après le palier : peut alors servir de diagnostic (si le buffer est mauvais à m_n, l'entraînement est probablement déficient).

G Preuves

Preuve 1. *Soit K une constante, g un modèle K-Lipschitz : $g = K \cdot f$, et f un modèle 1-Lipschitz :*

$$\sup_{\|g\|_L \leq K} \left| \mathbb{E}_{x \sim P_r}[g(x)] - \mathbb{E}_{x \sim P_g}[g(x)] \right|$$

$$= \sup_{\|g\|_L \leq K} \left| \mathbb{E}_{x \sim P_r}[K * f(x)] - \mathbb{E}_{x \sim P_g}[K * f(x)] \right|$$

$$= K * \sup_{\|f\|_L \leq 1} \left| \mathbb{E}_{x \sim P_r}[f(x)] - \mathbb{E}_{x \sim P_g}[f(x)] \right|$$

$$= K * W_1(P_r, P_g)$$